Integrating Blockchain and IPFS for Traceability, Integrity, and Ex-Post Auditing in Federated Learning

David Piotr Znojek[1*] and Francisco A. Pujol[1]

[1*]Department of Computer Technology and Computation, University of Alicante, Carretera de San Vicente del Raspeig s/n, San Vicente del Raspeig, 03690, Alicante, Spain.

*Corresponding author(s). E-mail(s): dawidznojek@gmail.com;
Contributing authors: fpujol@ua.es;

Abstract

Distributed training via Federated Learning (FL) preserves the privacy of sensitive edge data, but inherently lacks verifiable mechanisms to audit the authenticity and integrity of participant contributions, leaving systems vulnerable to model and data poisoning attacks with critical implications in domains such as Connected and Autonomous Vehicles (CAVs). This paper presents and evaluates a hybrid architecture that integrates a permissioned blockchain network based on Hyperledger Besu and IBFT 2.0 consensus with the Flower framework and off-chain content-addressable storage via IPFS. A Solidity smart contract implements a bidirectional cross-verification protocol between edge clients and the aggregation server using cryptographic commitments (IPFS content identifiers, CIDs) derived from neural network parameters. The viability of the framework is empirically validated on a physical distributed testbed training a convolutional classifier for vehicular hazard and road event detection under a rigorous multi-run protocol (15 complete independent executions totaling 450 federated rounds across five configurations). Quantitative results demonstrate that in a realistic distributed deployment, the blockchain auditability layer incurs a modest temporal overhead of only 9.5% (6.34 s per round), while maintaining a deterministic, invariant EVM gas profile per transaction (186,749 gas initial cold write, 169,649 gas steady state) and scaling with marginal impact (+0.39 s in median round time and +0.10 s in block confirmation delay) when expanding the IBFT 2.0 quorum from 4 to 7 validator nodes ($\boldsymbol{f = 1 \rightarrow f = 2}$). Furthermore, adversarial evaluation confirms deterministic interception of in-transit parameter tampering and unauthenticated injections at both client (149 ms) and server

(31 ms) levels. An ex-post audit protocol allows independent entities to mathematically reconstruct the canonical FedAvg aggregation directly from immutable Besu and IPFS commitments in 0.34 s with exact numerical correspondence ($\boldsymbol{\Delta} = \mathbf{0}$, CID match) and instantaneously detect deviations in 0.18 s. These findings confirm that the operational cost of decentralization, immutability, and non-repudiation is thoroughly viable for safety-critical distributed intelligent systems.

Keywords: Federated Learning, Blockchain, Hyperledger Besu, IBFT 2.0 Consensus, IPFS, Ex-Post Auditing, Cryptographic Integrity, Traceability, Autonomous Vehicles

1 Introduction

The rapid proliferation of Artificial Intelligence (AI) and Machine Learning (ML) has transformed critical sectors, including healthcare, finance, and intelligent transportation systems. However, training high-capacity models requires massive volumes of data, which frequently contain sensitive or proprietary information, such as high-resolution geolocation traces, user behavior patterns, or biometric identifiers. Under traditional centralized paradigms, gathering raw data into a single repository introduces severe privacy vulnerabilities, regulatory compliance challenges (such as the GDPR), and substantial computational and communication bottlenecks.

Federated Learning (FL) [1] emerged as a decentralized paradigm where distributed edge nodes iteratively train local model instances on their private datasets, transmitting only updated weight vectors or gradients to a central coordinator. Although FL avoids direct exposure of raw data, it introduces a structural security challenge: the absence of inherent trust among participating entities. In an unsupervised, distributed environment, malicious actors or compromised edge nodes can launch data poisoning, model poisoning, or backdoor insertion attacks [2] by submitting manipulated model updates. For instance, in Connected and Autonomous Vehicles (CAVs) [3], a subtle, coordinated perturbation in model parameters could lead the global classifier to misidentify critical road hazards or traffic signals (e.g., failing to detect a STOP sign), yielding catastrophic consequences.

To address these vulnerabilities, integrating Blockchain technology with Federated Learning (BCFL) [4] provides an immutable, decentralized, and verifiable ledger [5, 6]. By embedding a distributed ledger into the federated training workflow, parameter updates, aggregation rounds, and participant identities are cryptographically anchored via smart contracts, establishing an end-to-end, auditable, and non-repudiable record of all training contributions.

Nevertheless, this convergence introduces a critical system-level trade-off: **security versus operational performance**. While blockchain provides immutability and Byzantine fault tolerance, it inherently introduces network latency, consensus delays, transaction confirmation overhead, and additional computational resource demands.

In mission-critical edge computing environments [7], it is essential to answer a fundamental question: *what temporal and computational overhead must be incurred to achieve a more auditable and tamper-resistant federated training?*

1.1 Motivation and Prior Work

This work builds upon and extends two research lines conducted at the University of Alicante:

1. **Real-time vehicular event verification** [8]: demonstrated the feasibility of a private Hyperledger Besu network operating under Istanbul Byzantine Fault Tolerant (IBFT 2.0) consensus to log vehicular traffic alerts with sub-second latencies (0.32 to 0.82 s), outperforming human reaction times.
2. **FL-Blockchain architectural framework** [9]: conceptually proposed using smart contracts to record cryptographic hashes of model weights at each federated round.

The present study formally designs, implements, and empirically validates the convergence of these proposals on a physical distributed testbed, evaluating both cryptographic integrity guarantees and temporal system performance.

1.2 Main Contributions

The main contributions of this paper are summarized as follows:

- **Hybrid architecture design**: We formulate an end-to-end framework coupling a permissioned Hyperledger Besu blockchain network with the Flower federated learning framework [10] and off-chain, content-addressable distributed storage via IPFS.
- **Traceability and integrity via smart contracts**: We design and deploy a Solidity smart contract that orchestrates bidirectional cross-verification of models through content identifiers (IPFS CIDs, encapsulating SHA-256 multihashes of serialized artifacts), immutably anchoring each participant's contribution and the aggregated global model per round.
- **Empirical adversarial validation**: We subject the platform to deliberate weight-corruption attack vectors (both in-transit tampering from an untrusted server and unauthorized/poisoning injections from clients), validating that the bidirectional verification protocol deterministically intercepts integrity discrepancies and rogue models with low response latencies (31 to 149 ms).
- **Ex-post audit protocol and mathematical FedAvg verification**: We develop and evaluate an independent, off-band audit protocol enabling any consortium participant or third-party auditor to retrospectively verify aggregator fidelity. We demonstrate the ability to mathematically reconstruct canonical FedAvg aggregation directly from immutable commitments stored on Besu and IPFS, achieving exact numerical correspondence ($\Delta = 0$, CID match) in 0.34 s, while detecting unauthorized parameter deviations in 0.18 s.
- **Quantitative infrastructure benchmarking**: We deploy a convolutional traffic event classifier to serve as a representative workload for extracting reproducible

system-level metrics (temporal overhead, EVM gas consumption, and validator scaling), delimiting hyperparameter and model optimization as future work.
- **Operational feasibility analysis**: We analytically resolve the latency-versus-security dilemma, characterizing the operational regimes where blockchain logging overhead is thoroughly justified compared to unverified setups.

To facilitate scientific reproducibility and technology transfer, the complete framework source code, Solidity smart contracts, Hyperledger Besu Docker Compose deployment templates, and experimental evaluation scripts are openly available on GitHub: https://github.com/dpz6-ua/Blockchain_ML.

1.3 Paper Organization

The remainder of this paper is structured as follows: Section 2 reviews related work and theoretical foundations of Blockchain and Federated Learning. Section 3 details the proposed architecture and smart contract design. Section 4 describes the experimental testbed and dataset. Section 5 presents and discusses empirical performance, scalability, and security validation results. Finally, Section 6 concludes the paper and outlines future research directions.

2 Theoretical Foundations and Related Work

The proposed architecture converges three fundamental technological pillars: permissioned distributed ledgers with Byzantine Fault Tolerant (BFT) consensus, content-addressable distributed storage, and federated learning. Below, we synthesize their theoretical foundations and examine the critical limitations identified in the related literature.

2.1 Permissioned Blockchain and IBFT 2.0 Consensus

A blockchain is an immutable, distributed data structure where transactions are bundled into cryptographically linked blocks using collision-resistant hash functions (such as SHA-256) [6, 11]. Unlike public (permissionless) networks such as Bitcoin or Ethereum Mainnet, which suffer from high transaction latency and volatile monetary fees, industrial consortia predominantly adopt permissioned networks compatible with the Ethereum Virtual Machine (EVM) [12], such as Hyperledger Besu [13].

In these environments, compute-intensive (Proof-of-Work, PoW) or stake-based (Proof-of-Stake, PoS) mechanisms are replaced by Byzantine Fault Tolerant consensus algorithms. The **IBFT 2.0** (*Istanbul Byzantine Fault Tolerant*) protocol [14] organizes consensus around a predefined set of validator nodes that alternate the proposer role in a round-robin schedule. A proposed block is appended to the ledger only after a three-phase voting process (pre-prepare, prepare, commit) signed by a two-thirds supermajority of validators:

$$n \geq 3f + 1 \tag{1}$$

where n denotes the total number of validator nodes and f represents the maximum number of tolerated faulty or Byzantine participants.

IBFT 2.0 provides a pivotal advantage over probabilistic consensus models: **immediate deterministic finality**. Because the chain does not experience probabilistic forks, once a training round commitment or model update is sealed within a block, it achieves immediate finality and permanent immutability on the ledger. Furthermore, while the transaction gas price is set to zero in private consortia to eliminate monetary friction, the computational metric of *gas used* [15] remains an objective, deterministic measure of execution and storage overhead imposed on the EVM.

2.2 Off-Chain Distributed Storage (IPFS)

Directly persisting deep neural network parameters on a blockchain ledger is technically infeasible due to model artifact sizes (ranging from dozens to hundreds of megabytes), which would rapidly trigger severe EVM state bloat and demand prohibitive gas execution limits.

To overcome this bottleneck, we adopt the paradigm of **off-chain storage with on-chain commitment** utilizing the InterPlanetary File System (IPFS) [16, 17]. IPFS operates over a peer-to-peer (P2P) network based on content addressing and Merkle Directed Acyclic Graph (Merkle DAG) structures [17, 18]: files are partitioned into immutable chunks and addressed through a unique Content Identifier (CID) derived directly from their cryptographic hash. The global-scale empirical and architectural evaluation conducted by Trautwein et al. [18] demonstrates that its block-exchange protocol (Bitswap) and distributed hash table (Kademlia DHT) achieve decentralized content routing with minimal overhead. By decoupling weight artifact storage to IPFS and recording only the content identifier (IPFS CIDv0, encapsulating the SHA-256 multihash of the serialized weights) within the smart contract [19], subsequent downloads can be validated against the immutable ledger. This provides data deduplication, formal integrity verification, and distributed access at negligible on-chain execution cost, provided that operational availability and persistence are backed by content pinning or dedicated IPFS nodes.

2.3 Federated Learning and the FedAvg Algorithm

Federated Learning (FL) [1] preserves data privacy by decoupling local model optimization from central parameter aggregation. Among federated consensus mechanisms, **FedAvg** (*Federated Averaging*) serves as the foundational standard. At each communication round t, the aggregation coordinator broadcasts the current global weight vector $w^{(t)}$ to a selected set of K edge clients. Each client trains the model on its private partition $\mathcal{D}_k$ by minimizing a local loss function, yielding updated local parameter weights $w_k^{(t)}$. The server then computes the consolidated global model as a sample-weighted average based on local sample counts n_k:

$$w^{(t+1)} = \sum_{k=1}^{K} \frac{n_k}{n} w_k^{(t)}, \quad \text{where } n = \sum_{k=1}^{K} n_k \tag{2}$$

The **Flower** framework [10] offers a modular, efficient infrastructure to coordinate this federated lifecycle via bidirectional remote procedure calls over gRPC.

Table 1 Qualitative and architectural comparison against relevant Blockchain-FL and robustness frameworks in the literature.

Proposal / Framework	**Blockchain Consensus**	**Model Storage**	**Verification Protocol**	**Transactional Cost**	**Fault Tolerance**
Krum (Blanchard et al.) [20]	No Blockchain (Centralized)	Central server (SPOF)	Geometric selection (L_2)	None (No ledger)	Strict Byzantine ($f < \frac{N-2}{2}$)
BlockFL (Kim et al.) [21]	PoW (Slow / Inefficient)	On-Chain (Storage bottleneck)	Unidirectional	Volatile cryptocurrency	Probabilistic ($< 50\%$)
DeepChain (Weng et al.) [22]	PoW / PBFT with incentives	On-Chain (Encrypted gradients)	Homomorphic Enc. + MACs	Severe (Massive on-chain ciphertext)	Formal Byzantine ($f < N/3$)
AV-BFL (Pokhrel & Choi) [23]	Adaptive PoW / DPoS	On-Chain at RSUs	Base station verification	Wireless retransmissions	Constrained by vehicular channel
H-BCFL (Chai et al.) [24]	Hierarchical DPoS at RSUs	On-Chain at base stations	Reputation and knowledge audit	Edge clustering overhead	Dependent on RSU reliability
B5G-FL (Lu et al.) [25]	Permissioned (PBFT / BFT)	On-Chain at base stations	Base station consensus	DRL optimization	Formal Byzantine ($f < N/3$)
Vehicular BFL (Pujol et al.) [8]	IBFT 2.0 (Permissioned)	IoT alert logging	Traffic monitoring	Commercial gas-free	Formal Byzantine ($N \geq 3f + 1$)
BCFL-Ref (Goh et al.) [26]	PoW / PoA (Ethereum)	Hybrid Off-Chain (IPFS)	Unidirectional (No cross-verification)	Variable gas (Public/private net)	Formal Byzantine (Ethereum consensus)
Our Architecture	**QBFT / IBFT 2.0 (Besu)**	**Content-addressable storage (IPFS)**	**Bidirectional Cross-Verif. + Ex-Post Audit**	**Stable gas per transaction**	**Formal Byzantine ($f = 1 \rightarrow 2$)**

2.4 Vulnerabilities in FL and Related Work

Despite privacy advantages, conventional Federated Learning frameworks suffer from intrinsic vulnerabilities classified in recent security taxonomies and surveys [4, 27, 28]:

1. **Central server as a single point of failure and trust**: Existing frameworks offer no native mechanism to verify whether the central coordinator faithfully executes FedAvg or intentionally injects malicious biases into the global parameter vector $w^{(t+1)}$.
2. **Poisoning attacks and lack of non-repudiation**: Compromised edge clients can execute model poisoning or inject stealthy backdoors [2]. In the absence of an immutable, non-repudiable log linking cryptographic identities to parameter digests, attributing malicious updates or auditing the history of compromised models becomes impossible.

To address these vulnerabilities, researchers have increasingly investigated the hybridization of Blockchain and Federated Learning (BCFL) [4]. However, designing practical architectures for Connected and Autonomous Vehicles (CAVs/IoV) involves four fundamental engineering trade-offs:

1. On-Chain vs. Hybrid Off-Chain Storage and Computation: Pioneer architectures such as BlockFL [21] proposed exchanging full weight matrices directly over the blockchain ledger. Similarly, DeepChain [22] integrated decentralized auditing and economic incentives with homomorphic encryption (HE) of gradients, incurring extreme computational and on-chain storage bottlenecks. Other designs, such as FLChain [29], explored decentralized auditing using distributed Bloom filters, finding that serializing massive parameter tensors within blockchain transactions triggers acute EVM state bloat and degrades throughput. Consequently, the hybrid paradigm—leveraging content-addressable off-chain storage via IPFS anchored to on-chain cryptographic hashes [19, 26]—represents an efficient approach to prevent ledger bloat while ensuring scalability in distributed deployments.

2. Consensus Mechanisms and Latency in Wireless and Vehicular Networks: In autonomous driving and edge computing (5G and beyond), communication latency and resource constraints are primary bottlenecks [23, 25]. Proof-of-Work (PoW) protocols introduce multi-minute confirmation delays and probabilistic forks that are intolerable in safety-critical vehicular systems [21]. To optimize resource allocation and model convergence in cellular edge networks, Lu et al. [25] formulated a base-station permissioned blockchain utilizing deep reinforcement learning (DRL) for dynamic bandwidth allocation, while Pokhrel and Choi [23] developed analytical models characterizing vehicular wireless channel dynamics on consensus delay. In the Internet of Vehicles (IoV), Chai et al. [24] proposed a hierarchical architecture offloading aggregation and DPoS consensus to Roadside Units (RSUs), Qi et al. [30] combined permissioned blockchain with differential privacy for traffic flow prediction, and Abou El Houda et al. [31] devised collaborative intrusion detection schemes via BCFL. However, these frameworks depend heavily on the continuous availability and synchronization of fixed roadside infrastructure. Rather than relying on high-latency public networks or foregoing Byzantine fault tolerance, permissioned consortia running IBFT

2.0 / QBFT such as Hyperledger Besu [8] provide immediate deterministic finality and rigorous Byzantine resilience ($N \geq 3f + 1$) without monetary fee fluctuations.

3. Cryptographic Integrity vs. Statistical Robustness: Two complementary defensive tiers exist in federated learning security [27, 28]. On the one hand, decentralized immutable ledgers provide formal guarantees regarding in-transit artifact integrity, sender authentication, and non-repudiation. On the other hand, if a participating client deliberately manipulates its local training data while legitimately signing and registering the resulting weights on-chain (*data/model poisoning*), statistical aggregation defenses against anomalous gradients are essential. Literature has explored geometric filtering such as Krum [20], coordinate-wise trimming including Trimmed-Mean and Median [32], and hybrid defenses against complex collusions such as Bulyan [33]. Similarly, FLTrust [34] introduced trust bootstrapping by evaluating angular parameter similarity against a root dataset maintained by the coordinator. These two layers are complementary: on-chain non-repudiation unambiguously attributes each parameter update to a specific cryptographic identity, enabling post-facto forensic auditing and smart-contract-coordinated slashing once statistical anomalies are identified.

4. Aggregator Accountability (Black-Box Aggregation vs. Ex-Post Auditing): In verifiable aggregation literature without blockchain, pioneering works such as VerifyNet [35] and VeriFL [36] address verifiable aggregation using homomorphic commitments, zero-knowledge proofs, and secret sharing schemes. Although these mechanisms formally ensure that the central coordinator computes aggregation faithfully without reconstructing raw gradients, they impose heavy computational overhead and synchronous communication delays on edge clients during every federated round. Conversely, most BCFL architectures either assume the central coordinator is trustworthy or attempt to execute aggregation directly within the EVM at prohibitive gas and computational costs. To the best of our knowledge, existing frameworks lack a decoupled, deterministic mechanism that—without penalizing operational cycle times ($< 0.08\%$ overhead)—empowers any consortium participant or external auditor to reconstruct canonical FedAvg *ex-post* from immutable Blockchain and IPFS commitments, mathematically verifying global model fidelity without compromising local data privacy.

To contextualize the contributions of this research with respect to prior work, Table 1 presents a qualitative and architectural comparison. Unlike purely centralized statistical schemes (e.g., Krum), PoW-based architectures or massive on-chain storage paradigms (BlockFL, DeepChain), frameworks reliant on fixed RSU infrastructure, or unidirectional verification models lacking cross-validation (Goh et al. [26]), our architecture implements bidirectional cross-verification paired with an independent ex-post audit protocol over a permissioned consortium blockchain with immediate finality and stable gas costs per transaction.

3 Proposed System Architecture

To provide federated learning with robust traceability and Byzantine fault resilience, we propose a hybrid architecture decoupled into three operational planes [18, 19]:

the federated computation plane (Flower), the off-chain content-addressable storage plane (IPFS), and the on-chain consensus and immutability plane (Hyperledger Besu). Figure 1 illustrates the end-to-end interaction among these components.

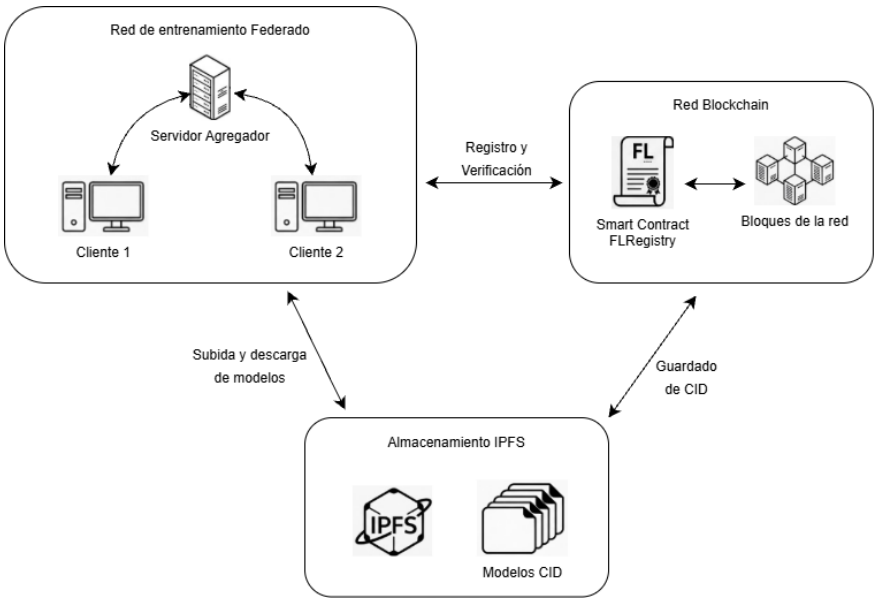


Fig. 1 Overall hybrid system architecture: decoupling between federated computation (Flower), off-chain parameter storage (IPFS), and immutable traceability logging (Hyperledger Besu with IBFT 2.0).

3.1 Smart Contract Design: FLRegistry

Governance and audit logic are encapsulated within the `FLRegistry` smart contract, implemented in Solidity (compiler version `0.8.10`). Its primary objective is to maintain a deterministic, chronological audit log of each federated round without storing massive numerical parameter matrices on-chain.

3.1.1 Data Structures and Persistence

The core entity within the contract is the `ModelRound` structure, defined as:

```
struct ModelRound {
    uint256 roundNum;
    string modelHash;
    address author;
    uint256 timestamp;
}
```

where `roundNum` indexes the federated iteration, `modelHash` stores the IPFS Content Identifier (CIDv0, encapsulating the SHA-256 multihash of the serialized model), `author` records the Ethereum public address of the sender (`msg.sender`), and `timestamp` captures the block validation timestamp.

Contract state is organized through two complementary mappings:

- `mapping(uint256 => ModelRound) public history`: sequentially records the global models aggregated by the coordinator at the conclusion of each round t.
- `mapping(uint256 => mapping(address => ModelRound)) public clientHistory`: indexes the individual local contributions submitted by each client in each round, enabling granular forensic auditing.

Additionally, state pointers `latestRound` and boolean flag `hasModel` are maintained to facilitate direct and constant-time read access without requiring iterative storage scans.

3.1.2 State-Modifying Primitives and Access Control

The contract exposes two state-modifying transactional functions that emit the decoupled `ModelUpdated` event:

- `registerClientModel(uint256 round, string memory hash)`: Invoked by each edge client k upon completing local training, immutably binding the CID of its contribution to its Ethereum address (`msg.sender`).
- `updateModel(uint256 round, string memory hash)`: Invoked operationally by the aggregation coordinator after executing the aggregation algorithm. To enforce consortium governance and prevent unauthorized participants or rogue nodes from submitting fraudulent global models, this function is guarded by the `onlyServer` access control modifier:

```
modifier onlyServer() {
    require(msg.sender == serverAddress,
            "Unauthorized sender");
    _;
}
```

  where `serverAddress` is initialized in the contract constructor with the authorized coordinator public address (which can be reassigned or governed via consortium multi-signature schemes). Consequently, the EVM deterministically reverts any unauthorized call to `updateModel`.

3.1.3 Read and Verification Primitives

To avoid imposing execution costs during routine checks, model verification queries are implemented as `view` functions, resolved locally at the JSON-RPC node without consuming gas:

- `getLastModel()`: Enables any client to retrieve the CID of the latest consolidated global model along with its validation timestamp.

- `getClientModel(uint256 round, address client)`: Enables the coordinator and external auditors to cross-check the CID reported by a client against its signed on-chain record.

3.1.4 Model-Agnostic Design and Generalization

The proposed framework is architecturally and operationally agnostic to neural network topology, data modality, and training framework. Because `FLRegistry` processes exclusively uniform 46-character string identifiers (the IPFS CIDs encapsulating the multihash digest of the serialized weights), EVM gas costs remain predictable and entirely decoupled from model parameter dimensions—whether the weights represent a shallow classifier, a deep convolutional network, or a Transformer architecture. Similarly, the IPFS storage plane treats serialized parameters as generic byte payloads, making this bidirectional verification protocol directly applicable across diverse modalities (tabular data, time series, acoustic signals, or computer vision) and federated optimization algorithms.

3.2 Operational Workflow and Bidirectional Cross-Verification

The federated lifecycle consists of a one-time bootstrap phase followed by an iterative protocol featuring strict bidirectional verification, detailed below:

3.2.1 Phase 0: Bootstrap and Baseline Model (Round 0)

Prior to concurrent training execution:

1. The Hyperledger Besu network operating under IBFT 2.0 consensus and the IPFS daemon are initialized.
2. `FLRegistry` is deployed, and its contract address and ABI are published in a shared configuration manifest.
3. The coordinator instantiates the initial model parameters $w^{(0)}$, serializes them, and uploads them to IPFS, obtaining the initial identifier CID_0.
4. The coordinator executes `updateModel(0,` CID_0`)`, establishing an auditable cryptographic baseline for all participants.

3.2.2 Phase 1: Downward Verification (Client Verifies Server)

Upon receiving the round start signal and candidate weights $w_{cand}^{(t)}$ via gRPC (Figure 2):

1. The client queries the blockchain by calling `getLastModel()`, retrieving the officially committed $\text{CID}_{reg}^{(t-1)}$.
2. The client fetches the referenced model artifact from IPFS and performs an exhaustive, element-wise numerical comparison across all tensor layers against $w_{cand}^{(t)}$ (verifying parametric equality within a floating-point tolerance of $\epsilon = 10^{-7}$).
3. If any discrepancy is detected between the delivered weights and the on-chain commitment, the client **aborts training immediately**, effectively neutralizing

split-view attacks where a compromised coordinator attempts to distribute biased models selectively to specific clients.

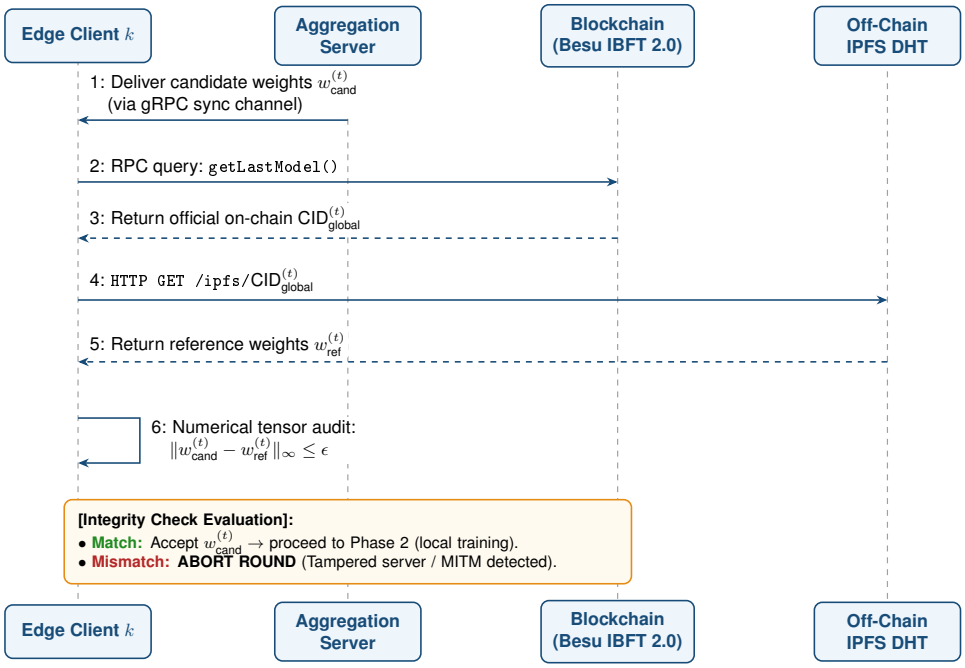


Fig. 2 Phase 1: Downward verification at the edge client: validating received candidate weights against the on-chain registered CID retrieved from IPFS.

3.2.3 Phase 2: Local Training and Contribution Commitment

If the downward verification check passes successfully (Figure 3):

1. The client performs local training on its private data partition.
2. It serializes the updated local weight vector $w_k^{(t)}$ and uploads it to IPFS, obtaining $\text{CID}_k^{(t)}$.
3. It broadcasts a signed transaction calling `registerClientModel`(t, $\text{CID}_k^{(t)}$) and awaits confirmation within an IBFT 2.0 block receipt.
4. It returns the parameter payload along with $\text{CID}_k^{(t)}$ and its public key to the Flower coordinator.

3.2.4 Phase 3: Upward Verification and Global Aggregation

Upon receiving client updates at the aggregation coordinator (Figure 4):

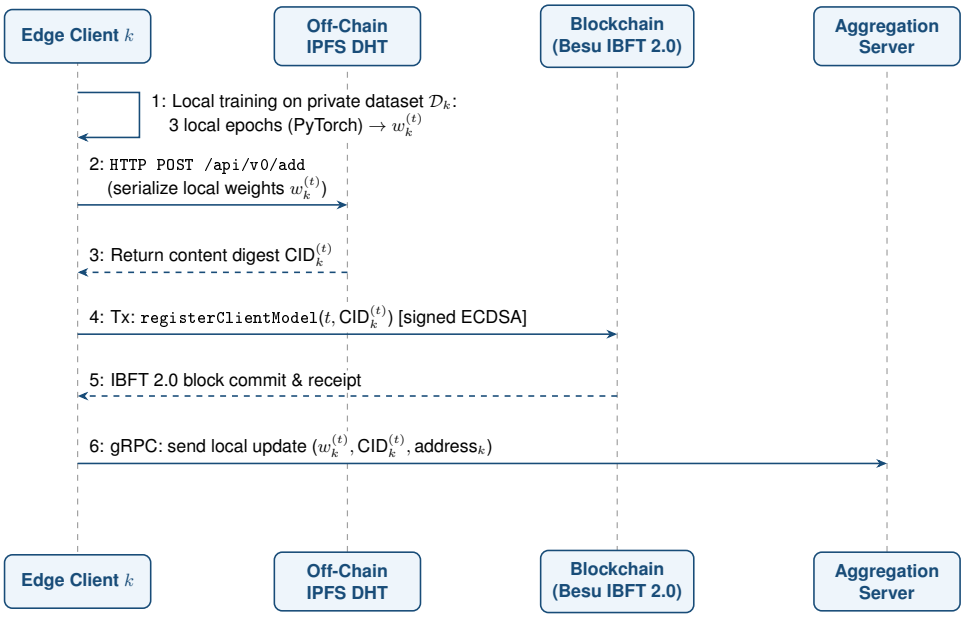


Fig. 3 Phase 2: Local training at the client, parameter upload to IPFS, and signed transaction commitment to Hyperledger Besu.

1. For each client k, the server receives parameter payload $w_k^{(t)}$, identifier $\text{CID}_{k,\text{gRPC}}^{(t)}$, and client address address_k.
2. **In-memory integrity verification:** The server deterministically computes the cryptographic digest of the received weight vector: $\text{CID}_{k,\text{calc}}^{(t)} = \text{Hash}(w_k^{(t)})$.
3. **On-chain verification and double cross-check:** The server queries `getClientModel(`t`, address`$_k$`)` from `FLRegistry`, retrieving $\text{CID}_{k,\text{onchain}}^{(t)}$, and enforces strict double equality:

$$\text{CID}_{k,\text{calc}}^{(t)} == \text{CID}_{k,\text{gRPC}}^{(t)} == \text{CID}_{k,\text{onchain}}^{(t)} \tag{3}$$

   This dual check counters two complementary threat vectors: if a malicious client commits a valid CID on Besu but transmits tampered weights over gRPC, or if a network adversary alters parameters in transit, $\text{CID}_{k,\text{calc}}^{(t)} \neq \text{CID}_{k,\text{onchain}}^{(t)}$; similarly, if the gRPC-reported CID does not match the signed transaction on Besu, verification fails immediately. In either case, the contribution is discarded prior to aggregation.
4. The server executes FedAvg aggregation exclusively over verified contributions:

$$w^{(t+1)} = \sum_{k \in \mathcal{K}_{valid}} \frac{n_k}{n_{valid}} w_k^{(t)} \tag{4}$$

   where n_k is the local sample count of client k and $n_{valid} = \sum_{k \in \mathcal{K}_{valid}} n_k$.

5. The server uploads the aggregated model to IPFS, obtaining $\text{CID}_{global}^{(t+1)}$, and anchors it on Besu by executing `updateModel`$(t+1, \text{CID}_{global}^{(t+1)})$. Because `updateModel` is restricted by `onlyServer`, no outside party can overwrite the consolidated global model.

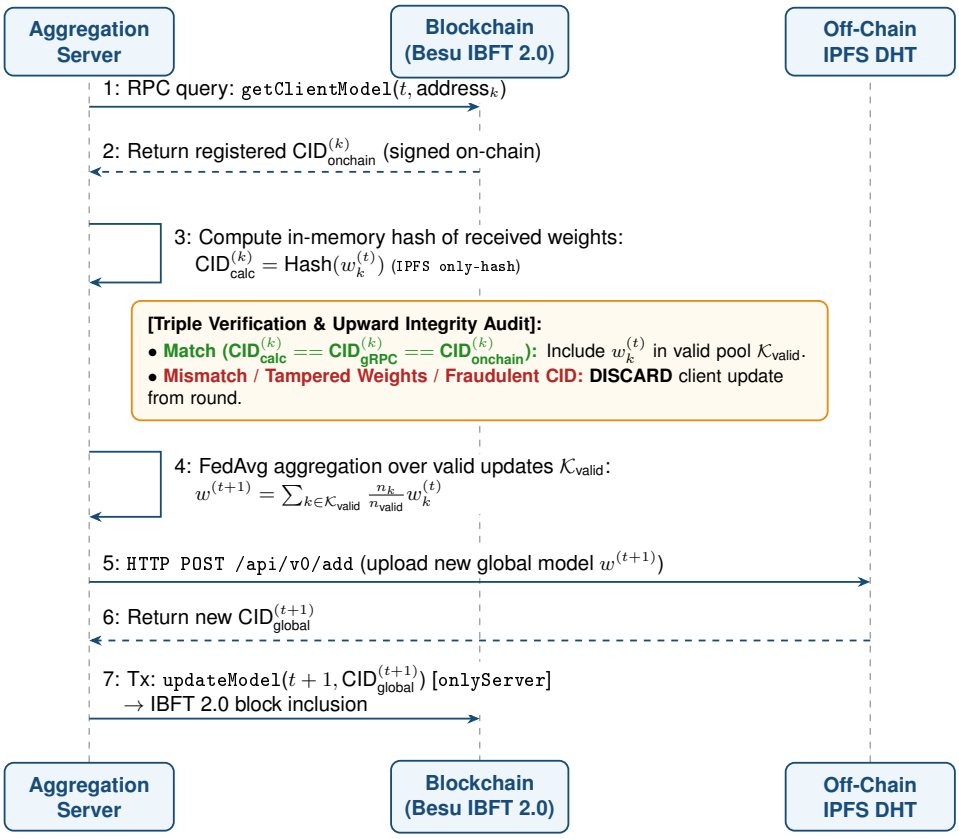


Fig. 4 Phase 3: Upward verification at the aggregation coordinator: tensor integrity validation, on-chain cross-check, FedAvg aggregation, and global round commitment.

3.3 Ex-Post Auditing and Verifiability of FedAvg

A distinctive architectural capability enabled by our hybrid decoupling is the ability to deterministically verify coordinator honesty retrospectively (*ex-post*). In conventional federated setups, the aggregation server operates as a trusted black box: participants must trust that aggregated weights $w^{(t+1)}$ faithfully represent a weighted average over received contributions, without arbitrary omissions, unauthorized weight injections, or stealthy backdoors.

In our decoupled architecture, every round t produces an immutable and reproducible audit trail linking the blockchain ledger to the content repository:

1. **Participant identification $\mathcal{K}_t$:** Because Solidity `mapping` structures are non-iterable by EVM design, active participants in round t are discovered deterministically by querying `ModelUpdated(round, modelHash, author)` event logs emitted during `registerClientModel` calls via standard RPC methods (`eth_getLogs`), or from the permissioned consortium whitelist.
2. **Local weight commitments:** Each participating client $k \in \mathcal{K}_t$ signs and registers its $\text{CID}_k^{(t)}$ in `clientHistory`, accessible via `getClientModel(`t`, address`$_k$`)`.
3. **Sample weighting factors n_k:** In balanced, homogeneous deployments (such as the evaluated experimental benchmark, where clients operate over equivalent IID partitions, $n_k = N/|\mathcal{K}_t|$), weighting factors are symmetric ($n_k / \sum n_j = 1/|\mathcal{K}_t|$). In heterogeneous settings (non-IID), scalar values n_k can be embedded within the IPFS metadata descriptor or included as an event parameter, allowing exact reconstruction without exposing raw training samples.
4. **Global model commitment:** The coordinator seals $\text{CID}_{\text{global}}^{(t+1)}$ in `history`, accessible via `getLastModel()` or direct round indexing.

From this immutable audit trail, any consortium member or independent third-party auditor can execute a deterministic aggregation check without accessing private datasets:

1. **Artifact retrieval:** The auditor queries Besu to retrieve active clients $\mathcal{K}_t$ and their committed $\text{CID}_k^{(t)}$, then downloads the binary parameter tensors $w_k^{(t)}$ from IPFS.
2. **Input integrity validation:** It verifies that for each client $k \in \mathcal{K}_t$, the cryptographic digest of the downloaded weights strictly matches the on-chain committed $\text{CID}_k^{(t)}$.
3. **Mathematical aggregation reconstruction:** It computes the canonical FedAvg aggregation:
$$\hat{w}^{(t+1)} = \sum_{k \in \mathcal{K}_t} \frac{n_k}{\sum_{j \in \mathcal{K}_t} n_j} w_k^{(t)} \tag{5}$$
4. **Global model cross-check:** It serializes $\hat{w}^{(t+1)}$ and computes its content identifier $\text{CID}_{audit}^{(t+1)} = \text{Hash}(\hat{w}^{(t+1)})$. Finally, it evaluates whether $\text{CID}_{audit}^{(t+1)} == \text{CID}_{global}^{(t+1)}$, registered in `FLRegistry`.

If $\text{CID}_{audit}^{(t+1)} \neq \text{CID}_{global}^{(t+1)}$, any coordinator deviation—whether due to biased weighting, participant omission, or unauthorized parameter tampering—is cryptographically, publicly, and non-repudiably proven. This verifiability transforms the centralized aggregator into a strictly accountable, provably honest orchestrator.

3.4 Threat Model and Security Scope

To clearly delineate the defensive guarantees provided by the framework, this section defines the threat model, distinguishing between vulnerabilities mitigated by design and those that lie outside our scope.

3.4.1 Mitigated Threats (In-Scope)

- **Man-in-the-Middle tampering**: A network adversary intercepting the gRPC channel who attempts to alter local parameter tensors sent to the server or global models delivered to clients. Discrepancies are intercepted immediately through element-wise numerical checks and on-chain CID verification.
- **Server equivocation / split-view attacks**: An untrusted or compromised coordinator attempting to deceive specific edge nodes by distributing divergent or altered global models (e.g., to facilitate active gradient inversion or localized performance degradation, as formalized by Carletti et al. [37]). Because clients accept only weights whose content identifier strictly matches the authoritative commitment in `FLRegistry`, divergent distribution attempts are blocked locally.
- **Unauthorized global model injection**: Attempts by rogue participants or external adversaries to commit unauthorized global models to the blockchain are rejected by the EVM through the `onlyServer` modifier.
- **Contribution repudiation**: Elliptic curve digital signatures (ECDSA secp256k1) underpinning Hyperledger Besu transactions prevent participants from repudiating submitted parameter updates.

3.4.2 Out-of-Scope Threats and System Boundaries

- **Semantic data and model poisoning with valid signatures**: If an adversary trains on poisoned datasets (e.g., label flipping or stealthy backdoors) but legitimately signs and registers the resulting model updates on Besu, cryptographic verification confirms provenance and integrity but cannot assess gradient statistical quality. Defense against semantic attacks requires pairing on-chain traceability with robust Byzantine aggregation rules (e.g., Krum [20], Median, Trimmed Mean) or forensic ex-post auditing of flagged contributions.
- **Private key compromise**: Cryptographic guarantees assume secure client key management. In industrial deployments, private keys must be secured via Hardware Security Modules (HSMs) or Trusted Execution Environments (TEEs).
- **Inference and gradient inversion attacks**: While our framework ensures integrity and provenance, it does not incorporate Differential Privacy (DP) or Secure Multiparty Computation (SMPC) to obfuscate shared gradients against reconstruction attacks [37].

3.4.3 Privacy and Weight Exposure on IPFS

On a public IPFS instance, any node knowing a model CID could retrieve parameter tensors and potentially attempt training data extraction. To protect model privacy in consortium deployments with stringent confidentiality requirements (e.g., autonomous fleets or healthcare), the framework supports two hardening mechanisms:

1. **Private IPFS Swarm**: Configuring the IPFS daemon in private swarm mode using a shared key (`swarm.key`) and setting `LIBP2P_FORCE_PNET=1`. This prevents unauthorized nodes from connecting to the P2P overlay or fetching storage blocks.
2. **Payload encryption at rest**: Encrypting serialized parameter vectors (e.g., using AES-256-GCM) prior to IPFS publication. Symmetric decryption keys can

be distributed via asymmetric public-key cryptography to authorized consortium members or managed through Attribute-Based Encryption (ABE).

4 Implementación y Entorno Experimental

Con el fin de aislar y evaluar cuantitativamente el sobrecoste introducido por la capa de trazabilidad y auditoría, se desplegó un banco de pruebas experimental representativo de un entorno de computación distribuida vehicular.

4.1 Infraestructura de Hardware y Software

La evaluación empírica se llevó a cabo utilizando dos estaciones físicas interconectadas en la misma red de área local (LAN), cuyas especificaciones técnicas se detallan en la Tabla 2:

- **Estación 1 (Servidor, Validadores e IPFS)**: Aloja el servidor agregador Flower, el daemon IPFS y la totalidad de los nodos validadores de la red permisionada Hyperledger Besu (los 4 validadores en las configuraciones E1 y E4, y los 7 validadores en E5). Cada validador se ejecuta de forma contenerizada como un servicio independiente mediante Docker Compose, interconectados a través de una red virtual en puente (*bridge*) que emula el mallado de consenso del consorcio vehicular y expone el puerto JSON-RPC del nodo validador principal como pasarela para el resto de entidades.
- **Estación 2 (Nodos de Borde / Clientes)**: Aloja y ejecuta los clientes de entrenamiento federado (*members* 2 y 3), los cuales se comunican con el servidor y la red blockchain a través de la conexión LAN.

Table 2 Especificaciones del banco de pruebas experimental.

Parámetro	**Estación 1 (Servidor/RPC)**	**Estación 2 (Clientes)**
Procesador	Intel Core i5-1135G7	Intel Core i7-9700
Frecuencia base	2,40 GHz	3,00 GHz
Núcleos / Hilos	4 núcleos / 8 hilos	8 núcleos / 8 hilos
Memoria RAM	7,6 GB (asignada en WSL2)	8,0 GB
Sistema Operativo	Ubuntu 22.04 LTS (WSL2)	Windows 11

El stack de software está compuesto por Hyperledger Besu operando con el algoritmo de consenso QBFT / IBFT 2.0 y un periodo de emisión de bloques fijado en 5 segundos (`blockperiodseconds: 5`), con soporte de transacciones privadas mediante Tessera y orquestación con Docker Compose; el daemon IPFS Kubo (`v0.32.1`); Python `3.12` con PyTorch [38] para la inferencia y optimización neuronal; Flower (`flwr`) [10] para la coordinación gRPC de la federación; y la biblioteca `web3.py` para la conectividad JSON-RPC con la EVM. Todos los contratos inteligentes, configuraciones de red y scripts de orquestación se encuentran disponibles en el repositorio público https://github.com/dpz6-ua/Blockchain_ML.

Las dos estaciones se encuentran comunicadas a través de un enlace de red de área local inalámbrico (Wi-Fi 802.11ac a 5 GHz), registrando una latencia media de ida y vuelta (*RTT ping*) de $3{,}8 \pm 1{,}1$ ms y un ancho de banda sostenido de 100 Mbps. El tamaño del modelo neuronal serializado (`NetCliente`) es de exactamente 1,95 MB en formato binario. Para favorecer un rendimiento ágil en la capa de almacenamiento distribuido durante el ciclo federado, la subida a IPFS se efectúa mediante la API HTTP en el endpoint `/api/v0/add?pin=false`, lo que permite generar el identificador CID e incorporar el bloque al repositorio local en milisegundos sin la sobrecarga síncrona del anclaje. Conviene puntualizar que, si bien la directiva `pin=false` optimiza la latencia inmediata entre rondas, no garantiza por sí misma la disponibilidad o persistencia indefinida de los artefactos frente a futuros procesos de recolección de basura (*garbage collection*) del daemon IPFS, requiriendo en despliegues productivos políticas de anclaje explícito o sincronización en un clúster IPFS persistente.

4.2 Modelo Neuronal y Conjunto de Datos Vehicular

Como caso de uso para simular un sistema de asistencia a la conducción en vehículos conectados y autónomos (CAV) [3], se adoptó el dataset de clasificación de eventos de tráfico del grupo CloudLab-AIA [9].

El problema consiste en una tarea de clasificación multiclase orientada a la detección en carretera con cuatro categorías:

1. Accidente vial (`accident`).
2. Congestión o tráfico denso (`dense_traffic`).
3. Condiciones meteorológicas adversas (`meteorology`).
4. Flujo de tráfico fluido (`sparse_traffic`).

El conjunto de datos comprende un total de 3.415 imágenes distribuidas en: meteorología adversa (1.027), tráfico fluido (947), tráfico denso (778) y accidentes (663). Las imágenes se preprocesan mediante reescalado a una resolución de 32×32 píxeles en tres canales cromáticos (RGB) y normalización estándar. Para preservar la comparabilidad estadística y eliminar sesgos de distribución, el conjunto de datos se reparte entre los $K = 2$ clientes según un esquema independiente e idénticamente distribuido (IID).

El modelo implementado (`NetCliente`) es una red neuronal convolucional (CNN) estructurada en dos bloques convolucionales consecutivos (con kernels de 3×3, función de activación ReLU y submuestreo por *Max Pooling*), seguidos de dos capas densas completamente conectadas (*fully connected*) que proyectan el vector de características hacia las 4 neuronas de salida. El entrenamiento se realiza mediante la función de pérdida de entropía cruzada (*Cross-Entropy Loss*) y el algoritmo de optimización Adam con una tasa de aprendizaje $\eta = 10^{-3}$, configurando un tamaño de lote (*batch size*) de 16 muestras en entrenamiento y 32 en evaluación, 3 épocas locales de entrenamiento por ronda y un horizonte total de 30 rondas federadas.

Es importante remarcar que el propósito primordial de este estudio no radica en diseñar un modelo convolucional óptimo ni en maximizar su precisión de clasificación, sino en utilizar la arquitectura de aprendizaje federado como un banco de pruebas realista para generar y analizar métricas cuantitativas del sistema (sobrecoste

temporal, escalabilidad y consumo de gas). La optimización del modelo (ajuste de hiperparámetros, mitigación del sobreajuste o empleo de arquitecturas más profundas) se plantea deliberadamente como trabajo futuro, actuando aquí la red neuronal como una carga de trabajo experimental representativa. De este modo, la arquitectura desacoplada resulta potencialmente aplicable a otros dominios de aprendizaje distribuido (tales como la monitorización médica, redes inteligentes o finanzas colaborativas), donde la lógica de auditoría en la Blockchain permanece análoga y únicamente el tiempo de transferencia y cálculo de hash off-chain en IPFS escala con el volumen de parámetros del modelo serializado.

4.3 Matriz de Configuraciones Experimentales

Para desglosar y desacoplar de forma analítica el impacto individual de cada componente, se diseñaron cinco configuraciones experimentales (E1 a E5), resumidas en la Tabla 3.

Table 3 Matriz de configuraciones experimentales evaluadas.

ID Exp.	Estaciones	Capa Blockchain	Validadores	Objetivo
E1	1	Sí	4	Sobrecarga BC local
E2	1	No	—	Baseline local
E3	2	No	—	Baseline distribuido
E4	2	Sí	4	Sobrecarga BC distribuida
E5	2	Sí	7	Escalabilidad IBFT 2.0

Este diseño metodológico permite efectuar comparaciones controladas de una sola variable:

- **Coste directo de Blockchain**: Comparación de E1 vs. E2 en entorno monomáquina, y de E4 vs. E3 en entorno distribuido en red.
- **Efecto de la distribución física**: Comparación de E2 vs. E3 (sin Blockchain) y de E1 vs. E4 (con Blockchain de 4 validadores).
- **Escalado del consenso BFT**: Comparación de E4 (4 validadores) vs. E5 (7 validadores), evaluando la penalización de latencia al elevar la tolerancia a fallos de $f = 1$ a $f = 2$.

5 Evaluación Experimental y Resultados

En esta sección se analiza cuantitativamente el comportamiento del sistema a lo largo del banco de pruebas experimental, respondiendo a las preguntas de investigación formuladas: el sobrecoste temporal de la capa de auditoría, el desacoplamiento de recursos físicos, la escalabilidad del consenso bizantino IBFT 2.0, la resiliencia empírica frente a ataques de adulteración y la previsibilidad del consumo de gas en la EVM. Con el objetivo de afianzar la solidez, robustez y reproducibilidad experimental, cada una de las cinco configuraciones experimentales (E_1 a E_5) fue evaluada a lo largo

de 3 ejecuciones independientes completas de 30 rondas cada una, totalizando un corpus de 15 ejecuciones y 450 rondas federadas ($N_{\text{total}} = 450$). Conviene enfatizar que la dinámica de convergencia del clasificador es estrictamente idéntica con o sin la presencia de Blockchain, ya que la capa de trazabilidad opera de forma desacoplada y no interfiere en los tensores ni en los gradientes de entrenamiento.

5.1 Formalización Matemática de Métricas de Rendimiento

Con el fin de aislar de forma unívoca la latencia impuesta por cada componente del sistema, el tiempo total por ronda en los nodos de borde (T_{client}) y en el servidor agregador (T_{server}) en la iteración t se define analíticamente como:

$$T_{\text{client}}^{(t)} = t_{\text{verif_desc}}^{(t)} + t_{\text{train}}^{(t)} + t_{\text{ipfs_up}}^{(t)} + t_{\text{tx_besu}}^{(t)} \tag{6}$$

$$T_{\text{server}}^{(t)} = t_{\text{wait}}^{(t)} + t_{\text{verif_asc}}^{(t)} + t_{\text{fedavg}}^{(t)} + t_{\text{ipfs_global}}^{(t)} + t_{\text{block_confirm}}^{(t)} \tag{7}$$

donde $t_{\text{verif_desc}}$ representa la comprobación local contra el CID de Besu; t_{train}, el tiempo de cálculo de gradientes con PyTorch; $t_{\text{ipfs_up}}$, la publicación binaria de pesos; $t_{\text{tx_besu}}$, la emisión y confirmación de `registerClientModel`; t_{wait}, la ventana de recepción gRPC; $t_{\text{verif_asc}}$, la validación en `getClientModel`; t_{fedavg}, la agregación matricial; y $t_{\text{block_confirm}}$, la confirmación del bloque IBFT 2.0 tras invocar `updateModel`.

Es crucial destacar la complementariedad entre ambas perspectivas temporales:

- **Tiempo de ciclo en el cliente (T_{client}, reportado en la Tabla 4):** Cuantifica la duración de la computación y comunicación local en los nodos de borde (vehículos), desde que reciben el modelo global hasta que sellan su contribución en la Blockchain. Esta métrica gobierna la autonomía operativa de los clientes y mantiene la comparabilidad directa con los estudios previos [9].
- **Tiempo de ronda extremo a extremo en el servidor (T_{server}, desglosado en la Tabla 5):** Engloba la totalidad del ciclo orquestado por el agregador central, incorporando la sincronización gRPC de todos los clientes concurrentes y la inclusión en bloque del nuevo modelo global mediante `updateModel` ($\approx$ 2,8–5,1 s adicionales por ronda).

Para medir el sobrecoste relativo (Δ_{overhead}) entre una configuración con seguridad E_{BC} y su respectiva línea base E_{base}, se emplea la mediana de las distribuciones:

$$\Delta_{\text{overhead}} = \frac{\text{Med}(T_{E_{\text{BC}}}) - \text{Med}(T_{E_{\text{base}}})}{\text{Med}(T_{E_{\text{base}}})} \times 100 \quad [\%] \tag{8}$$

En cuanto al rigor metodológico del análisis estadístico, el diseño experimental considera como unidades muestrales independientes las **15 ejecuciones completas** ($N_{\text{runs}} = 3$ repeticiones independientes por cada una de las 5 configuraciones). Las 30 rondas federadas consecutivas de cada ejecución no se presentan como observaciones independientes idénticamente distribuidas (i.i.d.), ya que cada iteración $t+1$ depende

intrínsecamente del estado paramétrico y las condiciones del sistema en la ronda t. Por consiguiente, las 450 rondas totales (90 por configuración) se emplean para el análisis descriptivo longitudinal y la visualización de trayectorias (Figura 5), mientras que la reproducibilidad inter-ejecución se cuantifica a través del Error Estándar (SE) entre las medias de las ejecuciones independientes ($SE = \sigma_{\text{runs}}/\sqrt{N_{\text{runs}}}$).

5.2 Visión General del Rendimiento Temporal Consolidado

La Tabla 4 sintetiza el tiempo de ciclo por ronda en los nodos de borde (T_{client}) a lo largo de las 15 ejecuciones independientes (3 por configuración, totalizando 90 rondas por entorno para análisis descriptivo continuo), reportando la mediana, media, desviación típica inter-ejecución (σ_{runs}) y error estándar ($SE = \sigma_{\text{runs}}/\sqrt{3}$). Asimismo, la Figura 5 refleja la serie temporal continua a lo largo de las rondas.

Table 4 Tiempo de ciclo por ronda en los nodos de borde (T_{client}) consolidado (15 ejecuciones independientes, 3 por configuración, 30 rondas/run). Los valores de σ_{runs} y SE cuantifican la dispersión y el error estándar entre las medias de las 3 ejecuciones independientes ($SE = \sigma_{\text{runs}}/\sqrt{3}$).

Exp.	Máq.	Configuración	Runs	Mediana (s)	Media (s)	σ_{runs} (s)	SE (s)
E_1	1	4 val · con BC	3	**85,42**	97,09	20,01	11,55
E_2	1	Baseline sin BC	3	**75,70**	76,45	1,65	0,95
E_3	2	Baseline sin BC	3	**66,62**	67,62	2,29	1,32
E_4	2	4 val · con BC	3	**72,96**	73,29	1,28	0,74
E_5	2	7 val · con BC	3	**73,35**	74,40	0,54	0,31

El análisis multiejecución revela diferencias clave en la estabilidad de cada entorno. En la configuración local E_1, la dispersión inter-ejecución entre las medias de las tres repeticiones es de $\sigma_{\text{runs}} = 20{,}01$ s ($SE = 11{,}55$ s), registrando una dispersión longitudinal acumulada a lo largo de las 90 rondas de $\sigma_{\text{pool}} = 27{,}64$ s. Esta mayor variabilidad se debe a que, en la primera ejecución en monomáquina, los procesos de entrenamiento convolucional compitieron directamente en CPU y memoria con los ocho contenedores de Docker que soportaban los nodos de Besu y el daemon de IPFS. Por el contrario, en el entorno distribuido (E_4 y E_5), la variabilidad inter-ejecución se reduce drásticamente ($\sigma_{\text{runs}} \in [0{,}54, 1{,}28]$ s y $SE \leq 0{,}74$ s; con dispersiones longitudinales contenidas en $\sigma_{\text{pool}} \leq 3{,}92$ s), demostrando una gran estabilidad y reproducibilidad operativa. Por este motivo, se adopta la **mediana** como métrica central para aislar el comportamiento intrínseco del sistema frente a fluctuaciones externas.

5.3 Sobrecoste Temporal Aislado de la Capa Blockchain

Para cuantificar con exactitud el coste temporal impuesto por la serialización de tensores, la publicación en IPFS, la firma ECDSA y el consenso IBFT 2.0, se comparan las configuraciones equivalentes con y sin capa de seguridad (Figura 6):

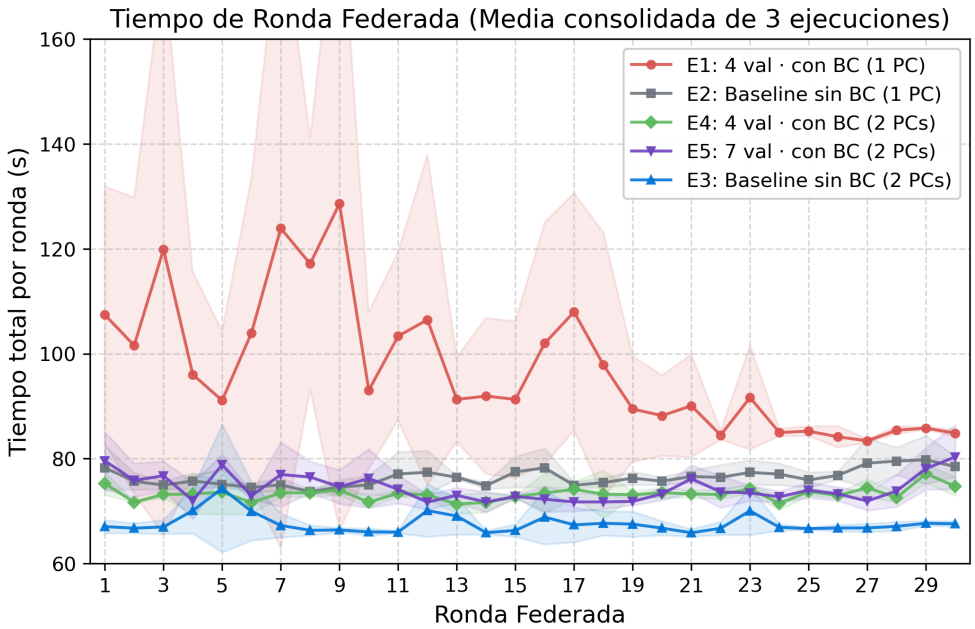


Fig. 5 Comparativa del tiempo de ciclo en clientes (T_{client}) a lo largo de las rondas en las cinco configuraciones experimentales.

- **Escenario Monomáquina (E_1 vs. E_2)**: El tiempo mediano pasa de 75,70 s a 85,42 s, representando un sobrecoste de **+9,72 s (+12,8%)**.
- **Escenario Distribuido (E_3 vs. E_4)**: El tiempo mediano se incrementa de 66,62 s a 72,96 s, lo que supone una sobrecarga de tan solo **+6,34 s (+9,5%)**.

Este hallazgo es de especial trascendencia para aplicaciones vehiculares y de borde: al desacoplar el entrenamiento de los clientes en una estación independiente, el impacto de la capa de auditoría e inmutabilidad se sitúa por debajo del 10%. Una penalización de apenas 6 segundos por ronda resulta ampliamente asumible frente al beneficio de contar con un registro descentralizado inmutable y auditable.

5.4 Efecto de la Distribución Física y Desglose por Fases

La comparación entre la arquitectura monomáquina y distribuida demuestra que la separación física de roles optimiza el rendimiento global (Figura 7): el tiempo de ronda se reduce tanto en el baseline sin Blockchain (de 75,70 s a 66,62 s; un descenso de -12,0%) como con Blockchain (de 85,42 s a 72,96 s; una reducción de -14,6%).

Para explicar rigurosamente este comportamiento, la Tabla 5 descompone la latencia media de cada fase interna del ciclo federado tanto en los clientes (Estación 2) como en el servidor (Estación 1), evidenciando la correspondencia con las métricas agregadas de la Tabla 4.

Los datos de la Tabla 5 confirman tres principios fundamentales:

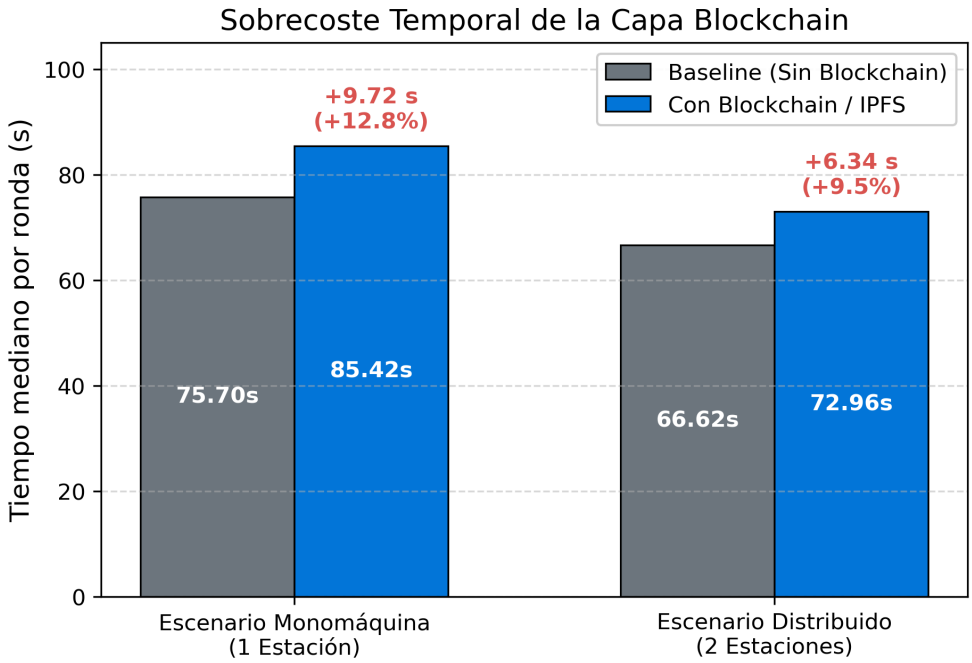


Fig. 6 Sobrecoste temporal inducido por la integración de la Blockchain en entornos local y distribuido.

Table 5 Desglose fino de la latencia media por fase en cliente (T_{client}) y servidor (T_{server}) (segundos).

	Fases Cliente (Estación 2)				**Fases Servidor (Estación 1)**			
Exp.	**Train**	**Verif.**	**IPFS**	**Tx Besu**	**Espera**	**Verif.**	**FedAvg**	**Bloque**
E_1	94,08	0,08	0,06	2,85	98,16	0,05	0,0065	5,14
E_2	76,45	0,00	0,00	0,00	77,02	0,00	0,0040	0,00
E_3	67,62	0,00	0,00	0,00	71,07	0,00	0,0036	0,00
E_4	66,22	1,68	2,43	2,84	79,54	0,06	0,0047	2,89
E_5	65,40	1,91	3,93	2,94	82,63	0,06	0,0079	2,99

1. **Desacoplamiento de cómputo:** En las configuraciones distribuidas (E_3, E_4, E_5), el tiempo dedicado exclusivamente a la optimización neuronal con PyTorch se mantiene estrictamente invariante ($\sim 65,4$–$67,6$ s), evitando la contención de recursos observada en E_1 ($94,08$ s).
2. **Sobrecarga de red y verificación acotadas:** La verificación remota vía RPC ($1,68$–$1,91$ s en cliente y ~ 5 ms en servidor), la transferencia IPFS ($2,43$–$3,93$ s) y el cómputo in-memory del multihash de pesos en el servidor (~ 52 ms en el código final, $< 0{,}08\%$ del ciclo) añaden una latencia mínima, mientras que el cálculo de FedAvg es despreciable (< 8 ms).

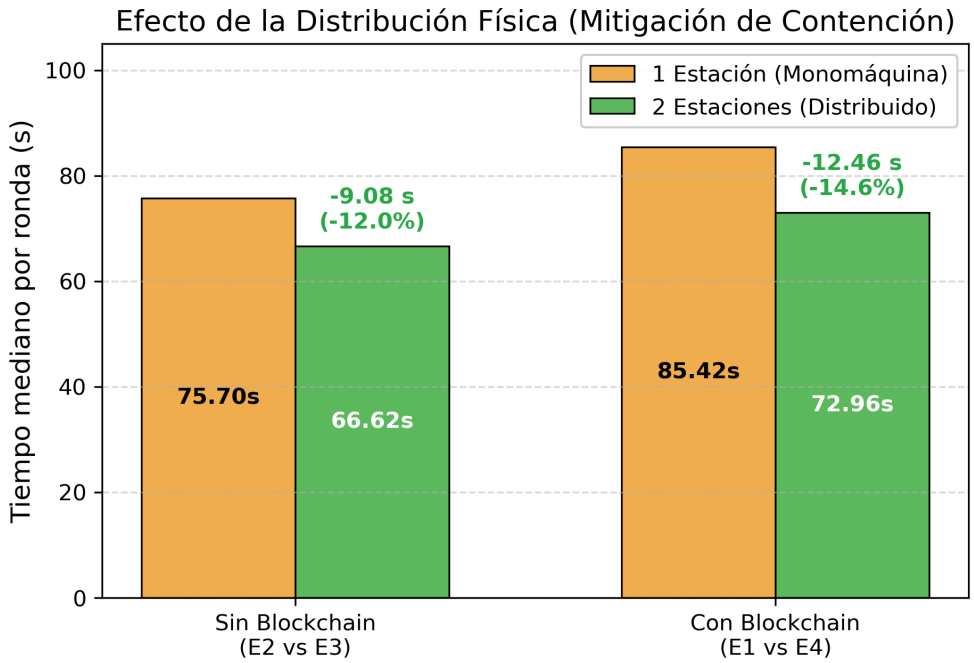


Fig. 7 Efecto de la distribución física de nodos sobre el tiempo de ciclo: mitigación de cuellos de botella de CPU/memoria.

3. **Correspondencia entre métricas (T_{client} y T_{server}):** La suma de las cuatro fases locales del cliente (*Train* + *Verif.* + *IPFS* + *Tx Besu*) totaliza con exactitud el tiempo de ciclo reportado en la Tabla 4 (por ejemplo, en E_4: 66,22 + 1,68 + 2,43 + 2,84 = 73,17 s, en plena coherencia con la media de 73,29 s). Paralelamente, la perspectiva del servidor refleja la duración global de la ronda en el agregador (T_{server}), donde el servidor aguarda la concurrencia de todos los clientes (*Espera*, 79,54 s) y añade la confirmación del nuevo modelo consolidado en la cadena (*Bloque*, 2,89 s), totalizando una latencia extremo a extremo de $\approx$ 82,49 s.

5.5 Escalado de Consenso Bizantino IBFT 2.0 (4 frente a 7 Validadores)

En redes permisionadas, la tolerancia a fallos bizantinos sigue el principio $N \geq 3f+1$, donde N es el conjunto de validadores y f el número máximo de nodos bizantinos o caídos:

- Red de 4 validadores (E_4): tolera $f = 1$ fallo; quórum de firmas necesario $2f+1 = 3$.
- Red de 7 validadores (E_5): tolera $f = 2$ fallos; quórum de firmas necesario $2f+1 = 5$.

Al comparar E_4 con E_5, la mediana por ronda apenas varía de 72,96 s a 73,35 s (+0,39 s), lo que evidencia que la latencia global del ciclo sigue dominada por el entrenamiento local.

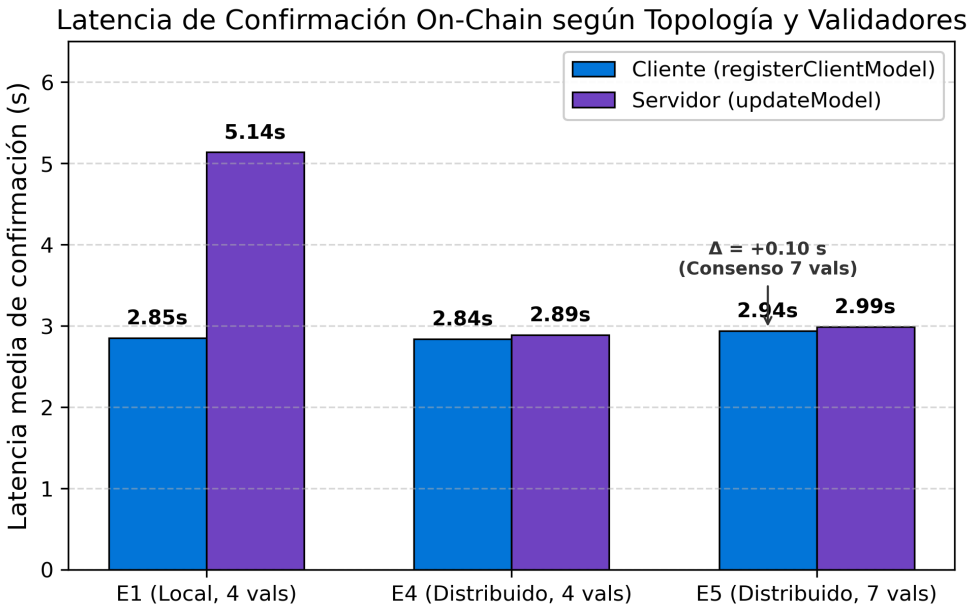


Fig. 8 Tiempo medio de confirmación de transacción según topología y número de validadores en Hyperledger Besu.

Al aislar específicamente el **tiempo de confirmación de las transacciones on-chain** (Figura 8), se cuantifica el impacto directo observado al ampliar el comité de consenso de 4 a 7 nodos validadores en el banco de pruebas evaluado:

- Transacción del cliente (`registerClientModel`): la latencia media pasa de 2,84 s en E_4 a 2,94 s en E_5 (+0,10 s).
- Transacción del servidor (`updateModel`): la latencia pasa de 2,89 s en E_4 a 2,99 s en E_5 (+0,10 s).

Este incremento simétrico de 100 ms refleja la sobrecarga medida empíricamente al ampliar el quórum de validación ($f = 1 \rightarrow 2$), confirmando que la mayor resiliencia bizantina del consorcio se absorbe holgadamente dentro del ciclo de cómputo local sin degradar la cadencia operativa del entrenamiento.

5.6 Robustez Adversarial y Verificación Experimental de Ataques

Para validar empíricamente que la arquitectura no solo registra transacciones en condiciones nominales, sino que neutraliza activamente vectores de ataque reales, se implementó una batería de ensayos adversariales controlados en el módulo `test_verificacion.py`. El objetivo es evaluar el comportamiento del sistema ante las dos principales amenazas bizantinas en aprendizaje federado no confiable: la equivocación del servidor agregador (*server equivocation*, en el flujo descendente) y un nodo participante envenenador (en el flujo ascendente).

5.6.1 Prueba 1: Verificación Descendente (Servidor Adulterado frente a Cliente Edge)

En un entorno federado no confiable, la distribución descendente de parámetros expone a los clientes a dos vectores de riesgo principales: la alteración de los tensores en tránsito por intermediarios de red (*Man-in-the-Middle*) y la bifurcación maliciosa o equivocación del servidor (*server equivocation*), donde un agregador deshonesto intenta distribuir modelos globales distintos o sesgados selectivamente a participantes específicos. Cabe destacar que la validez del modelo global on-chain queda estrictamente acotada a la entidad autorizada gracias al modificador de control de acceso `onlyServer` en el contrato `FLRegistry`, el cual revierte cualquier intento de registro por cuentas no autorizadas. Así, la rutina de verificación descendente implementada en el cliente protege de forma autónoma frente a discrepancias entre los pesos recibidos por gRPC y el compromiso sellado institucionalmente en la Blockchain:

1. **Generación y Sellado Oficial:** El servidor legítimo instancia el modelo base `NetCliente`, serializa los tensores mediante `pickle.dumps` y los publica en el daemon IPFS local a través del endpoint `/api/v0/add?pin=false`, obteniendo el CID oficial `QmQTpgejdzh...`. A continuación, el servidor emite una transacción firmada en la EVM invocando `updateModel(round, cid_oficial)` desde su cuenta autorizada (`0xfe3b55...`), quedando el compromiso criptográfico sellado en el **Bloque 80**.
2. **Inyección Adversarial y Simulación de Discrepancia:** Se emula un escenario adversarial donde se perturban los parámetros transmitidos al cliente inyectando ruido aditivo ($\mathbf{w}_{\text{cand}} = \mathbf{w}_{\text{oficial}} + 0{,}5$) y remitiendo este vector corrupto vía gRPC (simulando alteración en tránsito o un intento del servidor de suministrar una versión discrepante del modelo oficial).
3. **Comprobación Cruzada y Aborto Inmediato:** Al recibir la orden de entrenamiento, el cliente edge invoca la función de solo lectura `getLastModel()` en el Smart Contract mediante una llamada JSON-RPC para recuperar el CID sellado on-chain. Con este identificador descarga de forma paralela el artefacto binario de referencia desde la pasarela IPFS (`/ipfs/<CID>`) y ejecuta una comparación numérica tensorial elemento a elemento con cota de tolerancia de coma flotante (`np.allclose(p_recibido, p_ipfs, atol=1e-7)`).

Algoritmo 1. Verificación Descendente en el Cliente Edge

1. **Entrada:** Pesos candidatos $\mathbf{w}_{\text{cand}}$ remitidos vía gRPC por el servidor.
2. Consultar contrato en Besu: $\text{CID}_{\text{onchain}} \leftarrow$ `FLRegistry.getLastModel().cid`
3. Descargar binario de IPFS: $\mathbf{w}_{\text{ipfs}} \leftarrow$ `IPFS.get`$(\text{CID}_{\text{onchain}})$
4. **Si** $\exists l : \|\mathbf{w}_{\text{cand},l} - \mathbf{w}_{\text{ipfs},l}\|_\infty > \epsilon$ **entonces**
   - Registrar alerta: *"Discrepancia detectada en pesos recibidos"*
   - **ABORTAR** ronda localmente (bloquear backpropagation)
5. **Si no:** Aceptar $\mathbf{w}_{\text{cand}}$ y proseguir con el entrenamiento local.

La Figura 9 ilustra la telemetría real en tiempo de ejecución capturada en la consola del cliente. El proceso íntegro de consulta RPC a Besu, descarga y cotejo tensorial insumió tan solo **0,1486 segundos** (149 ms). Al constatar la discrepancia, el cliente detuvo de forma inmediata la ejecución, impidiendo la contaminación de su modelo local y salvaguardando los datos de entrenamiento privados del vehículo.

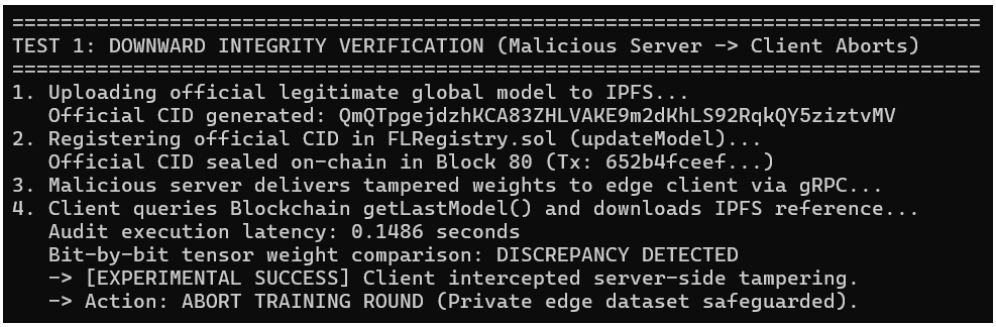


Fig. 9 Prueba 1: Registro de consola del nodo cliente detectando la adulteración del modelo frente al CID oficial sellado en el bloque

Por su parte, la Figura 10 aporta la evidencia inmutable en la cadena de bloques, capturada desde el explorador de la red Hyperledger Besu. En ella se observa el sellado de la transacción `updateModel` en el **Bloque 80**, emitida por la dirección del servidor (`0xfe3b557e...`) con hash criptográfico `0x973b0b...` y un consumo de gas de 167.460 en dicho ensayo previo (elevándose a 169.649 en la versión final con `onlyServer`, límite 500.000). Esta correspondencia unívoca entre el bloque minado y la verificación del cliente valida la consistencia del protocolo.

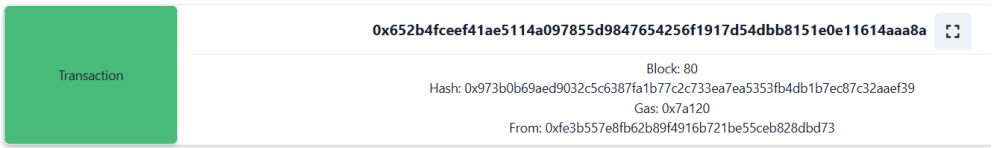


Fig. 10 Prueba 1: Confirmación en Hyperledger Besu del bloque 80, registrando la transacción oficial `updateModel` emitida por el servidor agregador.

5.6.2 Prueba 2: Verificación Ascendente (Cliente Envenenador frente a Servidor Agregador)

La segunda vulnerabilidad crítica consiste en la inyección de gradientes o pesos corruptos por parte de un nodo cliente malicioso (*model poisoning*). Para neutralizarla de raíz, el servidor no se limita a cotejar identificadores, sino que valida de forma cruzada la correspondencia biunívoca entre la carga útil recibida en memoria y el registro inmutable on-chain:

1. **Registro Legítimo On-Chain:** El nodo cliente honesto (cuenta 3, dirección `0xf17f52...`) publica sus pesos en IPFS y emite una transacción firmada con su clave privada invocando `registerClientModel(round, cid_legitimo)`, la cual queda consolidada por los validadores en el **Bloque 81**.
2. **Inyección Adversarial:** Se evalúa el comportamiento ante un intento de inyección donde se remite al servidor una actualización cuyos parámetros o metadatos no coinciden con el compromiso sellado on-chain (emulando tanto el envío de un identificador malicioso `QmClientFraud...` como el envío de pesos envenenados no correspondientes con el CID registrado).
3. **Auditoría Criptográfica y Descarte en FedAvg:** Antes de procesar la agregación matricial de la ronda, el servidor calcula el hash de los tensores recibidos en memoria ($\text{CID}^{(k)}_{\text{calc}}$) y ejecuta una llamada de solo lectura JSON-RPC `getClientModel(round, address)` sobre el contrato `FLRegistry` para obtener $\text{CID}^{(k)}_{\text{onchain}}$. Al constatar la discrepancia, la contribución es excluida de forma preventiva.

Algoritmo 2. Auditoría Criptográfica y RPC Ascendente en el Servidor Agregador

1. **Entrada:** Pesos $\mathbf{w}_k^{(t)}$ e identificador $\text{CID}^{(k)}_{\text{rep}}$ reportados por el cliente k con dirección addr_k.
2. Computar hash de los pesos recibidos: $\text{CID}^{(k)}_{\text{calc}} \leftarrow \text{Hash}(\mathbf{w}_k^{(t)})$
3. Consultar registro del cliente en Besu: $\text{CID}^{(k)}_{\text{onchain}} \leftarrow \texttt{getClientModel}(t, \text{addr}_k)$
4. **Si** $\text{CID}^{(k)}_{\text{rep}} \neq \text{CID}^{(k)}_{\text{onchain}}$ **o** $\text{CID}^{(k)}_{\text{calc}} \neq \text{CID}^{(k)}_{\text{onchain}}$ **entonces**
   - Registrar alerta: *"Discrepancia detectada / Inyección no autenticada"*
   - **DESCARTAR** contribución del cliente k del pool de FedAvg
5. **Si no:** Incorporar pesos $\mathbf{w}_k^{(t)}$ en la agregación global.

La Figura 11 documenta la salida en terminal del servidor agregador. La llamada JSON-RPC a la EVM tomó únicamente **0,0309 segundos** (31 ms). Al constatar que el CID no coincidía con la transacción firmada del cliente, el servidor excluyó al participante de la agregación de forma inmediata, preservando la convergencia y la integridad del modelo global.

La Figura 12 muestra el recibo del **Bloque 81** en el explorador Besu, donde consta la transacción `registerClientModel` emitida por la cuenta del cliente (`0xf17f5215...`) con hash `0x347899f...` y un consumo de gas de 160.200 (límite 200.000). Dado que el atacante remitió un CID ajeno al sellado en dicho bloque, la verificación cruzada interceptó la inyección adversarial gracias a la resistencia a colisiones de la función hash SHA-256 subyacente al CID de IPFS.

```
================================================================================
TEST 2: UPWARD INTEGRITY AUDIT (Malicious Client -> Server Discards from FedAvg)
================================================================================
1. Honest Client (0xf17f5215...) registers on-chain...
   Legitimate CID sealed on-chain in Block 81 (Tx: 47d2bfbb2c...)
2. Malicious client attempts injecting fraudulent CID to server via gRPC:
   Reported payload CID: QmClientFraudNotRegisteredInBesu22222222
3. Aggregation server audits reported CID via getClientModel(round, address)...
   RPC query latency:       0.0309 seconds
   On-chain registered CID: QmClientLegitModelWeightsCID1111111111111111
   ECDSA signature & Hash match: NO (FRAUD DETECTED)
   -> [EXPERIMENTAL SUCCESS] Aggregator intercepted poisoning attempt.
   -> Action: DISCARD CLIENT CONTRIBUTION from FedAvg pool.
   -> Global model convergence protected; Byzantine resilience verified.
```

Fig. 11 Prueba 2: Registro de consola del servidor agregador auditando vía RPC el CID del cliente y descartando la contribución fraudulenta

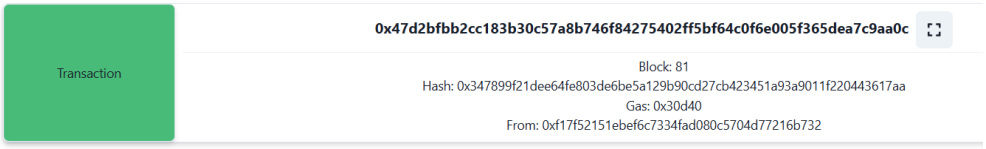


Fig. 12 Prueba 2: Confirmación forense en Hyperledger Besu del bloque 81, registrando la transacción legítima `registerClientModel` del nodo cliente.

5.6.3 Prueba 3: Auditoría Ex-Post y Verificabilidad Independiente de FedAvg

Para validar la tercera garantía de integridad formalizada en la Sección 3.3, se evaluó experimentalmente la capacidad de un agente auditor externo o regulador para fiscalizar *a posteriori* y de forma autónoma la fidelidad de la agregación ejecutada por el servidor central. Esta prueba permite auditar matemáticamente si el agregador ha respetado la regla canónica de agregación sin introducir sesgos, omisiones ni alteraciones paramétricas no autorizadas.

El ensayo se ejecutó sobre la Ronda 2 de un entrenamiento real de aprendizaje federado con dos clientes edge concurrentes conectados a la red Hyperledger Besu (contrato `0x42699A76...`) e IPFS. La Tabla 6 sintetiza la cadena de custodia completa de dicha ronda, correlacionando de forma unívoca las cuentas emisoras, las transacciones selladas en los bloques consecutivos 73, 74 y 75, los identificadores multihash IPFS y las latencias registradas.

El protocolo de auditoría operó conforme a las siguientes etapas:

1. **Extracción e indexación on-chain:** De forma desacoplada y fuera de banda, el auditor identifica dinámicamente a los clientes participantes $\mathcal{K}_2$ consultando los registros de eventos `ModelUpdated` de la Ronda 2 en Besu (descubriendo las cuentas del Cliente 2 y Cliente 3) y recupera sus compromisos criptográficos sellados (`QmNh3TLS...` y `QmS5ViVf...`), junto con el modelo global reportado por el servidor (`QmWF7tJ1...`).

Table 6 Cadena de custodia y trazabilidad cruzada on-chain/IPFS durante la Ronda 2 y su fiscalización ex-post.

Entidad	Dirección Ethereum	Evento / Método	Bloque Besu	Latencia	CID IPFS	Estado / Resultado
Cliente 3	`0xf17f5215...`	`registerClient-Model`	Bloque 73	0,886 s	`QmS5ViVf...`	Confirmado on-chain
Cliente 2	`0x62730609...`	`registerClient-Model`	Bloque 74	4,311 s	`QmNh3TLS...`	Confirmado on-chain
Servidor	`0xfe3b557e...`	`updateModel` (FedAvg)	Bloque 75	4,709 s	`QmWF7tJ1...`	Modelo global sellado
Auditor	*Externo (Off-chain)*	Reconstrucción FedAvg	Consulta 73–75	0,339 s	`QmWF7tJ1...`	$\Delta = 0{,}000000$ **(MATCH)**

2. **Descarga y reconstrucción determinista:** El auditor descarga los tres artefactos binarios desde la pasarela IPFS (2,08 MB por modelo, totalizando 6,24 MB recuperados en 0,200 s acumulados [0,096 s, 0,044 s y 0,060 s]) y ejecuta la agregación matricial canónica de FedAvg:

$$\mathbf{w}_{\text{recon}}^{(2)} = \frac{1}{2}\mathbf{w}_{\text{cli2}}^{(2)} + \frac{1}{2}\mathbf{w}_{\text{cli3}}^{(2)} \tag{9}$$

   generando el hash IPFS del modelo resultante mediante el algoritmo multihash SHA-256.
3. **Verificación tensorial y criptográfica:** Se cotejan capa por capa los parámetros del modelo reconstruido frente al modelo global sellado por el servidor en el Bloque 75.

Como refleja la telemetría en tiempo de ejecución capturada en la Figura 13, la auditoría integral (autodescubrimiento dinámico de participantes vía eventos Besu, consulta RPC de compromisos, descarga de 6,24 MB de IPFS, reconstrucción tensorial y verificación exhaustiva de las 8 capas de la red convolucional) requirió únicamente **0,3391 segundos**. La discrepancia paramétrica en todas las capas convolucionales y completamente conexas fue estrictamente nula ($\Delta = 0{,}000000\text{e}+00$), derivando en un identificador CID idéntico carácter a carácter al registrado en el Bloque 75 (`CID Equality: MATCH`), ratificando de forma incontrovertible la concordancia de la agregación.

Asimismo, se evaluó la sensibilidad del mecanismo activando el modo adversarial simulado (`--simulate-tampering`). Al inyectar una perturbación aditiva deliberada de +0,1 en los pesos del servidor, el auditor detectó la desviación de forma instantánea en tan solo **0,1805 segundos** ($\Delta = 1{,}000000\text{e}-01$), registrando una alerta formal de discrepancia no conforme (`AUDIT FAILED: DISCREPANCY DETECTED`).

5.6.4 Síntesis de Validación y Alcance de Seguridad

La Tabla 7 sintetiza las métricas cuantitativas obtenidas a lo largo de las tres pruebas experimentales de integridad y auditoría.

Estos ensayos constituyen una validación funcional determinista de las primitivas de intercepción criptográfica implementadas en el marco: ante una alteración de los

Table 7 Validación experimental del marco de integridad y auditoría frente a escenarios de manipulación y verificación ex-post.

Escenario de Prueba	**Entidad Defensora / Auditora**	**Acción Preventiva / Resultado**	**Latencia**	**Integridad de Pesos**
Manipulación en Tránsito / Servidor No Fidedigno	Cliente (Edge)	Intercepción determinista / Aborto preventivo	0,1486 s	Sin contaminación local
Inyección No Autenticada / Cliente No Registrado	Servidor Agregador	Intercepción determinista / Descarte en FedAvg	0,0309 s	Agregación preservada
Auditoría FedAvg Ex-Post	Auditor Independiente	Reconstrucción matemática / Verificación fidedigna	0,34 s	Fidelidad exacta ($\Delta = 0$, CID MATCH)

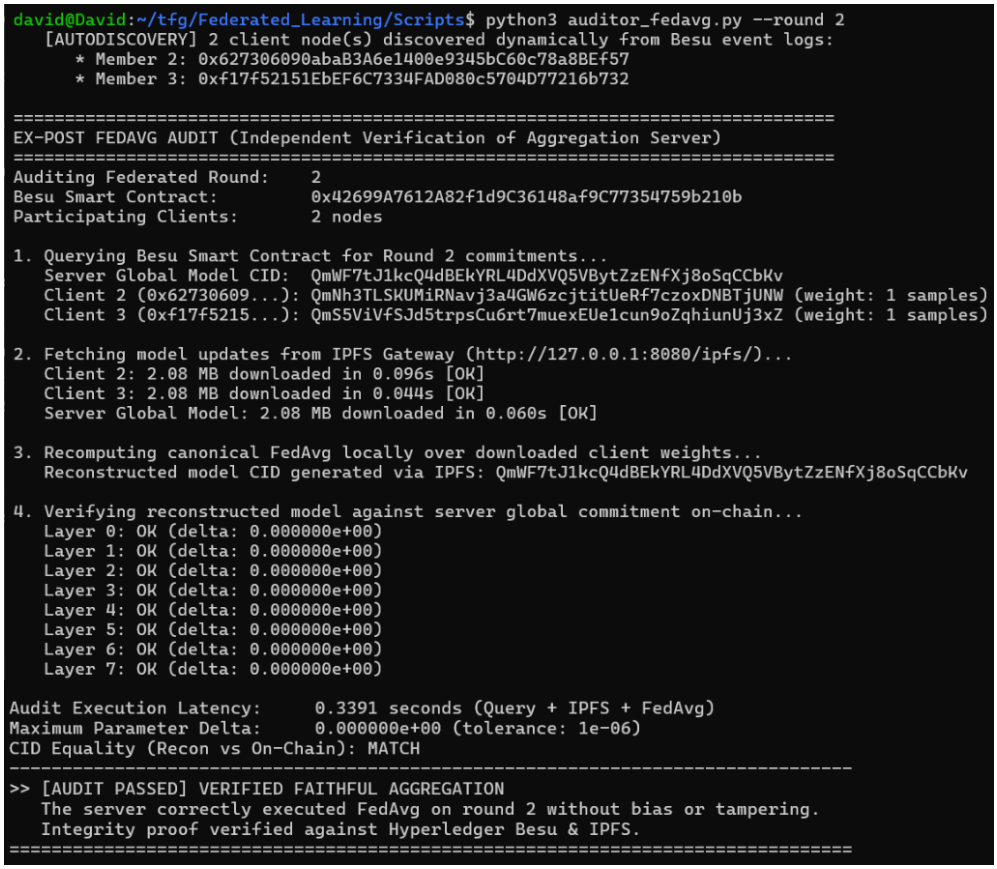


Fig. 13 Prueba 3: Registro de consola del auditor independiente reconstruyendo la agregación FedAvg de la Ronda 2 a partir de Besu e IPFS, verificando fidelidad estricta ($\Delta = 0{,}000000$) y coincidencia de CID.

parámetros en tránsito o la ausencia de un compromiso inmutable previamente sellado y firmado en la cadena, el protocolo intercepta la anomalía de manera determinista con tiempos de respuesta en torno a la décima de segundo (149 ms en el cliente, 31 ms en el servidor y 0,34 s en la auditoría ex-post de FedAvg), impidiendo que los pesos discordantes contaminen los cómputos locales o globales.

Tal como se formalizó en el modelo de amenazas (Sección 3.4), conviene remarcar los límites operacionales de este esquema defensivo: la doble verificación en cadena actúa con eficacia contra la adulteración directa de parámetros en tránsito (*tampering*), la emisión no autorizada a nombre de terceros mediante autenticación asimétrica ECDSA y la inyección de modelos disidentes no respaldados on-chain. No obstante, si un nodo malicioso genera una actualización perjudicial mediante manipulación local de sus datos de entrenamiento (*data poisoning* o *label flipping*) [39] y la firma legítimamente con su clave privada en Besu, el compromiso criptográfico coincidirá formalmente aunque la semántica del modelo esté contaminada. La neutralización de esta tipología de ataques semánticos requiere complementar la trazabilidad con filtros

de agregación estadística bizantina (como Krum [20], Median o Trimmed-Mean) o mediante la auditoría ex-post de las contribuciones registradas posibilitada por nuestro diseño (Sección 3.3).

5.7 Consumo y Previsibilidad del Gas en la EVM

La Figura 14 ilustra el consumo de gas computacional de las operaciones registradas a lo largo de las rondas.

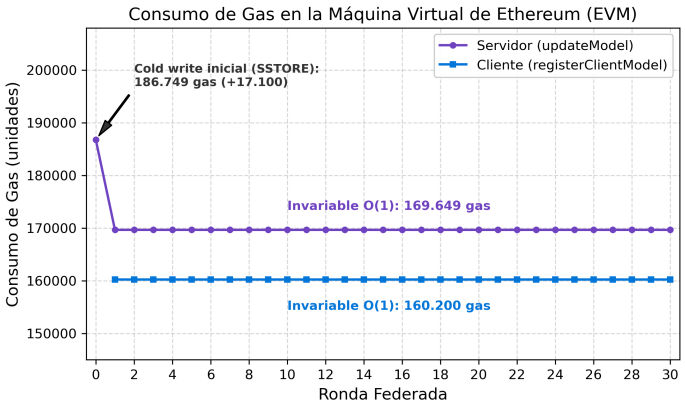


Fig. 14 Consumo de gas en la Máquina Virtual de Ethereum por tipo de transacción a lo largo de las 30 rondas.

El perfil de consumo resulta rigurosamente constante en todas las iteraciones:

- `registerClientModel`: exactamente **160.200 unidades de gas**.
- `updateModel`: exactamente **169.649 unidades de gas**.

Conviene destacar que este consumo refleja la versión definitiva del contrato inteligente `FLRegistry`: el despliegue del contrato con el modificador de control de acceso `onlyServer` en `updateModel` introduce una comprobación de identidad en la EVM (`SLOAD` y salto condicional sobre `serverAddress`) que sitúa el coste de ejecución en exactamente 169.649 unidades de gas (un sobrecoste acotado de 2.189 unidades de gas, +1,3% respecto a una versión abierta sin control de acceso), manteniéndose rigurosamente constante y predecible a lo largo de todas las rondas operativas. La única excepción observable corresponde a la ronda 0 del servidor, que demandó **186.749 unidades de gas**. Dicha discrepancia (+17.100 unidades) responde al coste de la instrucción `SSTORE` de la EVM al escribir por primera vez en posiciones de memoria previamente vacías (*cold write*) y modificar el valor del flag booleano `hasModel`.

La invariabilidad del consumo valida formalmente el diseño desacoplado: al almacenar en el contrato únicamente cadenas de longitud uniforme (los identificadores CIDv0 de IPFS de 46 caracteres que encapsulan el multihash SHA-256 y metadatos acotados), el coste de gas se mantiene estable por transacción e independiente tanto del tamaño del modelo neuronal como del número acumulado de rondas, asegurando un consumo de recursos acotado y predecible en redes de consorcio.

5.8 Auditoría de Inmutabilidad On-Chain y Trazabilidad Forense

Para verificar empíricamente la correspondencia unívoca entre los eventos del ciclo de vida de Flower y la persistencia inmutable en la cadena de bloques, la Tabla 8 recoge una muestra representativa de los recibos de transacción (*transaction receipts*) extraídos de la telemetría en tiempo de ejecución, correlacionada directamente con la visualización en el explorador de bloques de la red de consorcio (Figura 15).

Table 8 Muestra de trazabilidad y auditoría forense on-chain.

Bloque	Entidad	Evento / Método	Dirección Emisora	Prefijo Tx Hash	Gas EVM
8	Servidor	Despliegue `FLRegistry`	`0xfe3b557e...`	`0x939eb46c...`	1.999.000
13	Servidor	`updateModel`	`0xfe3b557e...`	`0xa18e3bb4...`	169.649
14	Cliente 2	`registerClientModel`	`0x62730609...`	`0xc1c7402f...`	160.200
765	Servidor	`updateModel` (Ronda 23)	`0xfe3b557e...`	`0x4ceabb7a...`	169.649
783	Servidor	`updateModel` (Ronda 24)	`0xfe3b557e...`	`0xb9cb75f5...`	169.649
801	Servidor	`updateModel` (Ronda 25)	`0xfe3b557e...`	`0xdf69295f...`	169.649

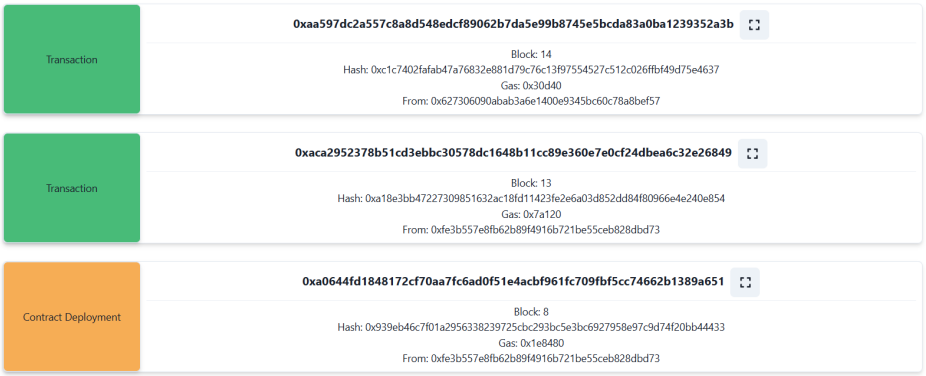


Fig. 15 Auditoría de inmutabilidad on-chain: visualización en el explorador Besu del despliegue del contrato `FLRegistry` (Bloque 8) por el servidor (`0xfe3b55...`) junto a transacciones operativas 13 y 14.

Cada identificador de bloque y hash criptográfico actúa como prueba forense no repudiable, confirmando la viabilidad de la arquitectura para auditorías regulatorias en sistemas vehiculares y de misión crítica.

6 Conclusiones y Trabajo Futuro

En este artículo se ha presentado y validado experimentalmente un marco integral orientado a proporcionar trazabilidad, integridad y auditoría criptográfica en Aprendizaje Federado, integrando Hyperledger Besu con IPFS y el framework Flower.

6.1 Conclusiones Principales

Los resultados obtenidos permiten extraer las siguientes conclusiones clave:

- **Viabilidad técnica y sobrecoste temporal asumible**: La integración de trazabilidad en un despliegue distribuido añade una sobrecarga de tan solo el **9,5%** (+6,34 s por ronda, de 66,62 s a 72,96 s), demostrando que la auditabilidad de extremo a extremo es plenamente viable en IA distribuida sin degradar la cadencia operativa.
- **Eficacia y rapidez de la doble verificación cruzada**: Se mitiga la dependencia del servidor central como punto único de confianza. Mediante el cotejo de CIDs de IPFS frente al registro on-chain, los clientes quedan protegidos frente a modelos adulterados (abortando en 0,149 s) y el agregador descarta de inmediato contribuciones no firmadas (en 0,031 s), garantizando intercepción determinista.
- **Eficiencia y predictibilidad on-chain**: El desacoplamiento con IPFS asegura un coste de gas constante por transacción (160.200 gas para clientes y 169.649 gas para el servidor), evitando saturar el estado de la EVM con total independencia del tamaño del modelo.
- **Escalabilidad del consenso BFT**: Incrementar los nodos validadores de 4 a 7 (elevando la tolerancia a fallos bizantinos de $f = 1$ a $f = 2$) apenas eleva la latencia de confirmación de transacciones en 0,10 s (100 ms) y la mediana de ronda en tan solo 0,39 s, absorbiéndose dentro del cómputo local.
- **Fiscalización ex-post y verificabilidad de FedAvg**: Se supera la limitación del servidor central como caja negra mediante auditoría externa independiente. Se validó empíricamente la reconstrucción de la agregación canónica en tan solo 0,34 s con fidelidad estricta ($\Delta = 0{,}000000$), detectando manipulaciones en 0,18 s sin acceder a datos privados locales.

6.2 Limitaciones y Trabajo Futuro

Como líneas de trabajo futuro se plantean:

1. **Optimización del modelo de IA**: Al centrarse este estudio en la infraestructura del sistema, se plantea como trabajo futuro incorporar parada temprana (*early stopping*), regularización (Dropout, normalización) y redes más profundas para maximizar la generalización.
2. **Mecanismos de reputación y penalización on-chain**: Explotar el historial inmutable de contribuciones para articular algoritmos de puntuación o penalización (*slashing*) económica/lógica de nodos maliciosos recurrentes.
3. **Agregación bizantina avanzada**: Complementar el esquema con estrategias de agregación robustas frente a envenenamiento no evidente (como Krum [20] o medias recortadas) coordinadas desde el propio contrato.
4. **Entornos no-IID y despliegue vehicular V2X**: Evaluar la robustez del framework bajo distribuciones heterogéneas y desplegar nodos en hardware embarcado (*edge computing*, como NVIDIA Jetson) con conectividad V2X.

5. **Gestión criptográfica de claves**: Sustituir el esquema de cuentas actual por módulos de seguridad de hardware (HSM) o almacenes seguros de secretos tipo Key Vault para un despliegue industrial.
6. **Gobernanza descentralizada y comités rotatorios**: Extender el control de acceso del servidor (`onlyServer`) hacia esquemas de gobernanza multifirma (DAO/multisig) o comités rotatorios de agregación para descentralizar la autorización institucional de los modelos globales.

En síntesis, la arquitectura validada demuestra que la convergencia entre Blockchain y Aprendizaje Federado no solo es factible, sino que establece una base metodológica viable para el despliegue auditable y confiable de inteligencia artificial en entornos críticos cooperativos.

Declarations.

- **Funding**: Not applicable.
- **Conflict of interest**: The authors declare that they have no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.
- **Data and materials availability**: The datasets and runtime telemetry logs generated during the empirical evaluation are available upon reasonable request.
- **Code availability**: Orchestration scripts, Solidity smart contracts, Docker Compose deployment templates, and independent audit modules are openly accessible at https://github.com/dpz6-ua/Blockchain_ML.

References

[1] McMahan, B., Moore, E., Ramage, D., Hampson, S., Arcas, B.A.y.: Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS) (2017)

[2] Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., Shmatikov, V.: How to backdoor federated learning. In: Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS). Proceedings of Machine Learning Research, vol. 108, pp. 2938–2948. PMLR, ??? (2020). https://doi.org/10.48550/arXiv.1807.00459

[3] Chellapandi, V.P., Yuan, L., Wang, Z., Żak, S.H.: Federated learning for connected and automated vehicles: A survey of existing approaches and challenges. IEEE Transactions on Intelligent Vehicles **9**(1), 119–137 (2024) https://doi.org/10.1109/TIV.2023.3332675

[4] Qu, Y., Uddin, M.P., Gan, C., Xiang, Y., Gao, L., Yearwood, J.: Blockchain-enabled federated learning: A survey. ACM Computing Surveys **55**(4), 1–35 (2022) https://doi.org/10.1145/3524104

[5] Nakamoto, S.: Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf. Accedido en 2026 (2008)

[6] Zheng, Z., Xie, S., Dai, H.-N., Chen, X., Wang, H.: Blockchain challenges and opportunities: a survey. International Journal of Web and Grid Services **14**(4), 352–375 (2018) https://doi.org/10.1504/IJWGS.2018.095647

[7] Liu, S., Liu, L., Tang, J., Yu, B., Wang, Y., Shi, W.: Edge computing for autonomous driving: Opportunities and challenges. Proceedings of the IEEE **107**(8), 1697–1716 (2019) https://doi.org/10.1109/JPROC.2019.2915983

[8] Pujol, F.A., Mora, H., Ramírez, T., Rocamora, C., Bedón, A.: Blockchain-based framework for traffic event verification in smart vehicles. IEEE Access **12**, 9251–9266 (2024) https://doi.org/10.1109/ACCESS.2024.3352738

[9] CloudLab-AIA: FL-Blockchain: Federated Learning & Blockchain for Vehicular Event Recognition. https://github.com/cloudlab-aia/FL-Blockchain. Repositorio en GitHub. Accedido: 2026-06-15 (2026)

[10] Beutel, D.J., Topal, T., Mathur, A., Qiu, X., Fernandez-Marques, J., Gao, Y., Sani, L., Li, K.H., Parcollet, T., Gusmão, P.P.B., Lane, N.D.: Flower: A friendly federated learning research framework. arXiv preprint arXiv:2007.14390 (2020)

[11] National Institute of Standards and Technology: FIPS PUB 180-4: Secure Hash Standard (SHS). Technical report, U.S. Department of Commerce (2015)

[12] Buterin, V.: Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform. https://ethereum.org/en/whitepaper/. Accedido en 2026 (2014)

[13] Hyperledger Besu: Hyperledger Besu Documentation. https://docs.besu-eth.org/private-networks. Accedido en 2026

[14] Moniz, H.: The istanbul bft consensus algorithm. arXiv preprint arXiv:2002.03613 (2020)

[15] Ethereum Foundation: Gas and fees. https://ethereum.org/en/developers/docs/gas/. Accedido en 2026

[16] Benet, J.: Ipfs - content addressed, versioned, p2p file system. arXiv preprint arXiv:1407.3561 (2014)

[17] Daniel, E., Tschorsch, F.: IPFS and friends: A qualitative comparison of next generation peer-to-peer data networks. IEEE Communications Surveys & Tutorials **24**(1), 31–52 (2022) https://doi.org/10.1109/COMST.2022.3143147

[18] Trautwein, D., Raman, A., Tyson, G., Castro, I., Scott, W., Schubotz, M., Gipp,

B., Psaras, Y.: Design and evaluation of IPFS: A storage layer for the decentralized web. In: Proceedings of the ACM SIGCOMM 2022 Conference (SIGCOMM '22), pp. 739–752. ACM, ??? (2022). https://doi.org/10.1145/3544216.3544232

[19] Zhao, Y., Zhao, J., Jiang, L., Tan, R., Niyato, D., Li, Z., Lyu, L., Liu, Y.: Privacy-preserving blockchain-based federated learning for IoT devices. IEEE Internet of Things Journal **8**(3), 1817–1829 (2021) https://doi.org/10.1109/JIOT.2020.3017377

[20] Blanchard, P., El Mhamdi, E.M., Guerraoui, R., Stainer, J.: Machine learning with adversaries: Byzantine tolerant gradient descent. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 119–129 (2017)

[21] Kim, H., Park, J., Bennis, M., Debbah, M.: Blockchained on-device federated learning. IEEE Communications Letters **24**(6), 1279–1283 (2019) https://doi.org/10.1109/LCOMM.2019.2921755

[22] Weng, J., Weng, J., Zhang, J., Li, M., Zhang, Y., Luo, W.: Deepchain: Auditable and privacy-preserving deep learning with blockchain-based incentive. IEEE Transactions on Dependable and Secure Computing **18**(5), 2438–2455 (2021) https://doi.org/10.1109/TDSC.2019.2952332

[23] Pokhrel, S.R., Choi, J.: Federated learning with blockchain for autonomous vehicles: Analysis and design challenges. IEEE Transactions on Communications **68**(8), 4734–4746 (2020) https://doi.org/10.1109/TCOMM.2020.2990686

[24] Chai, H., Leng, S., Chen, Y., Zhang, K.: A hierarchical blockchain-enabled federated learning algorithm for knowledge sharing in internet of vehicles. IEEE Transactions on Intelligent Transportation Systems **22**(7), 3975–3986 (2021) https://doi.org/10.1109/TITS.2020.3002712

[25] Lu, Y., Huang, X., Zhang, K., Maharjan, S., Zhang, Y.: Blockchain and federated learning for 5g beyond. IEEE Network **35**(1), 219–225 (2020) https://doi.org/10.1109/MNET.011.1900598

[26] Goh, E., Kim, D.-Y., Lee, K., Oh, S., Chae, J.-E., Kim, D.-Y.: Blockchain-enabled federated learning: A reference architecture design, implementation, and verification. IEEE Access **11**, 145747–145762 (2023) https://doi.org/10.1109/ACCESS.2023.3345360

[27] Bouacida, N., Mohapatra, P.: Vulnerabilities in federated learning. IEEE Access **9**, 63229–63249 (2021) https://doi.org/10.1109/ACCESS.2021.3075203

[28] Jere, M.S., Farnan, T., Koushanfar, F.: A taxonomy of attacks on federated learning. IEEE Security & Privacy **19**(2), 20–28 (2021) https://doi.org/10.1109/MSEC.2020.3046736

[29] Bao, X., Su, C., Xiong, Y., Huang, W., Xiang, Y.: FLChain: A blockchain for auditable federated learning with trust and incentive. In: Proceedings of the 5th International Conference on Big Data Computing and Communications (BIGCOM), pp. 151–159. IEEE, ??? (2019). https://doi.org/10.1109/BIGCOM.2019.00030

[30] Qi, Y., Hossain, M.S., Nie, J., Li, X.: Privacy-preserving blockchain-based federated learning for traffic flow prediction. Future Generation Computer Systems **117**, 328–337 (2021) https://doi.org/10.1016/j.future.2020.12.003

[31] Abou El Houda, Z., Moudoud, H., Brik, B., Khoukhi, L.: Blockchain-enabled federated learning for enhanced collaborative intrusion detection in vehicular edge computing. IEEE Transactions on Intelligent Transportation Systems **25**(7), 7661–7672 (2024) https://doi.org/10.1109/TITS.2024.3351699

[32] Yin, D., Chen, Y., Kannan, R., Bartlett, P.L.: Byzantine-robust distributed learning: Towards optimal statistical rates. In: Proceedings of the 35th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 80, pp. 5650–5659. PMLR, ??? (2018)

[33] El Mhamdi, E.M., Guerraoui, R., Rouault, S.: The hidden vulnerability of distributed learning in byzantium. In: Proceedings of the 35th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 80, pp. 3521–3530. PMLR, ??? (2018)

[34] Cao, X., Fang, M., Liu, J., Gong, N.Z.: FLTrust: Byzantine-robust federated learning via trust bootstrapping. In: Proceedings of the 28th Network and Distributed System Security Symposium (NDSS) (2021). https://doi.org/10.14722/ndss.2021.24434

[35] Xu, G., Li, H., Liu, S., Yang, K., Lin, X.: Verifynet: Secure and verifiable federated learning. IEEE Transactions on Information Forensics and Security **15**, 911–926 (2020) https://doi.org/10.1109/TIFS.2019.2929409

[36] Guo, X., Liu, Z., Li, J., Gao, J., Hou, B., Dong, C., Baker, T.: Verifl: Communication-efficient and fast verifiable aggregation for federated learning. IEEE Transactions on Information Forensics and Security **16**, 1736–1751 (2021) https://doi.org/10.1109/TIFS.2020.3043139

[37] Carletti, V., Foggia, P., Mazzocca, C., Parrella, G., Vento, M.: On the detectability of active gradient inversion attacks in federated learning. In: Proceedings of the 2026 IEEE Symposium on Security and Privacy (S&P), pp. 1931–1950. IEEE, ??? (2026). https://doi.org/10.1109/SP59124.2026.00085

[38] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshteyn, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai,

J., Chintala, S.: PyTorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 32, pp. 8024–8035 (2019)

[39] Tolpegin, V., Truex, S., Gursoy, M.E., Liu, L.: Data poisoning attacks against federated learning systems. In: Computer Security – ESORICS 2020. Lecture Notes in Computer Science, vol. 12308, pp. 480–501. Springer, ??? (2020). https://doi.org/10.1007/978-3-030-58951-6_24